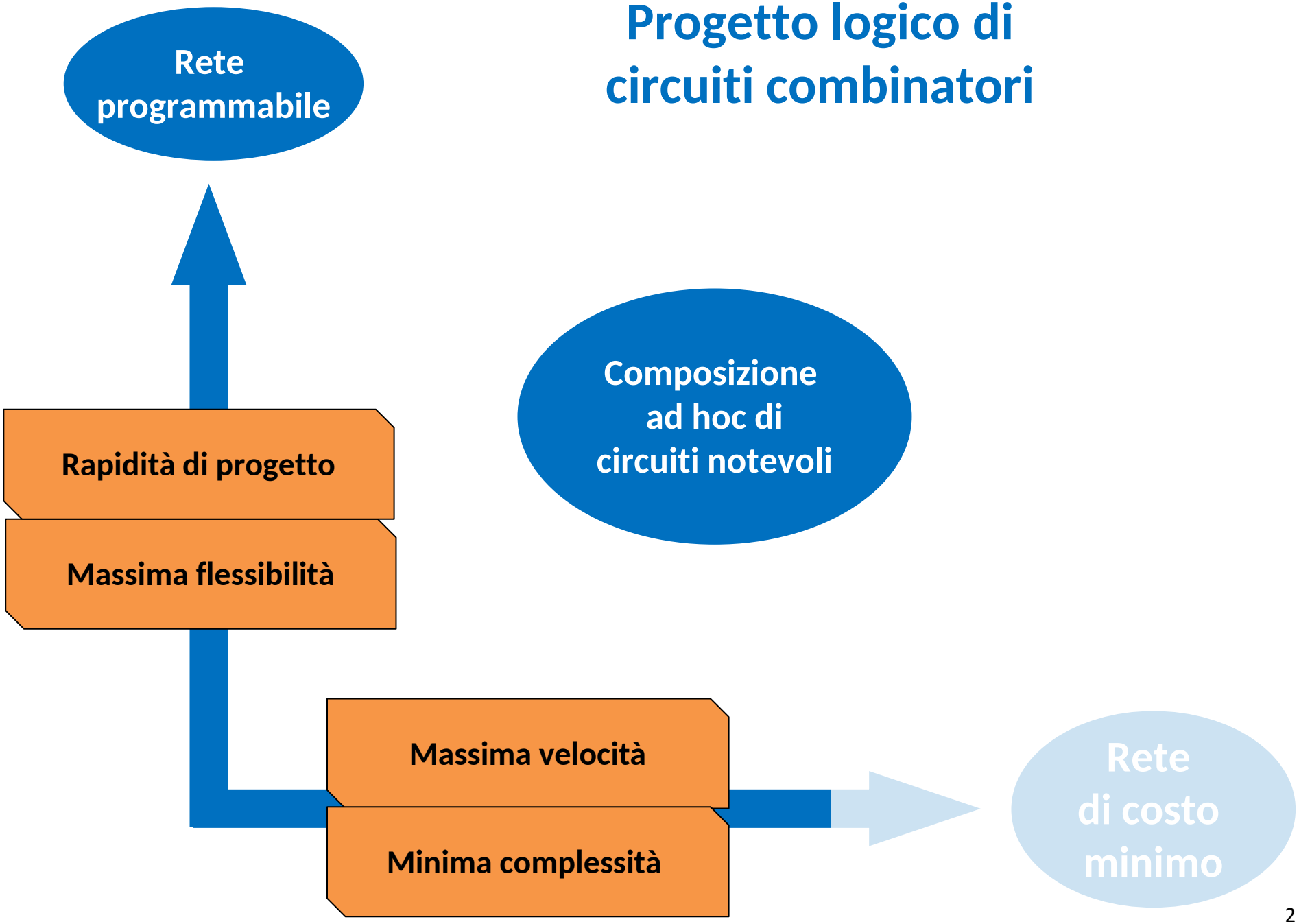
Sintesi tramite decoder, multiplexer e ROM

Reti Logiche T

Ingegneria Informatica

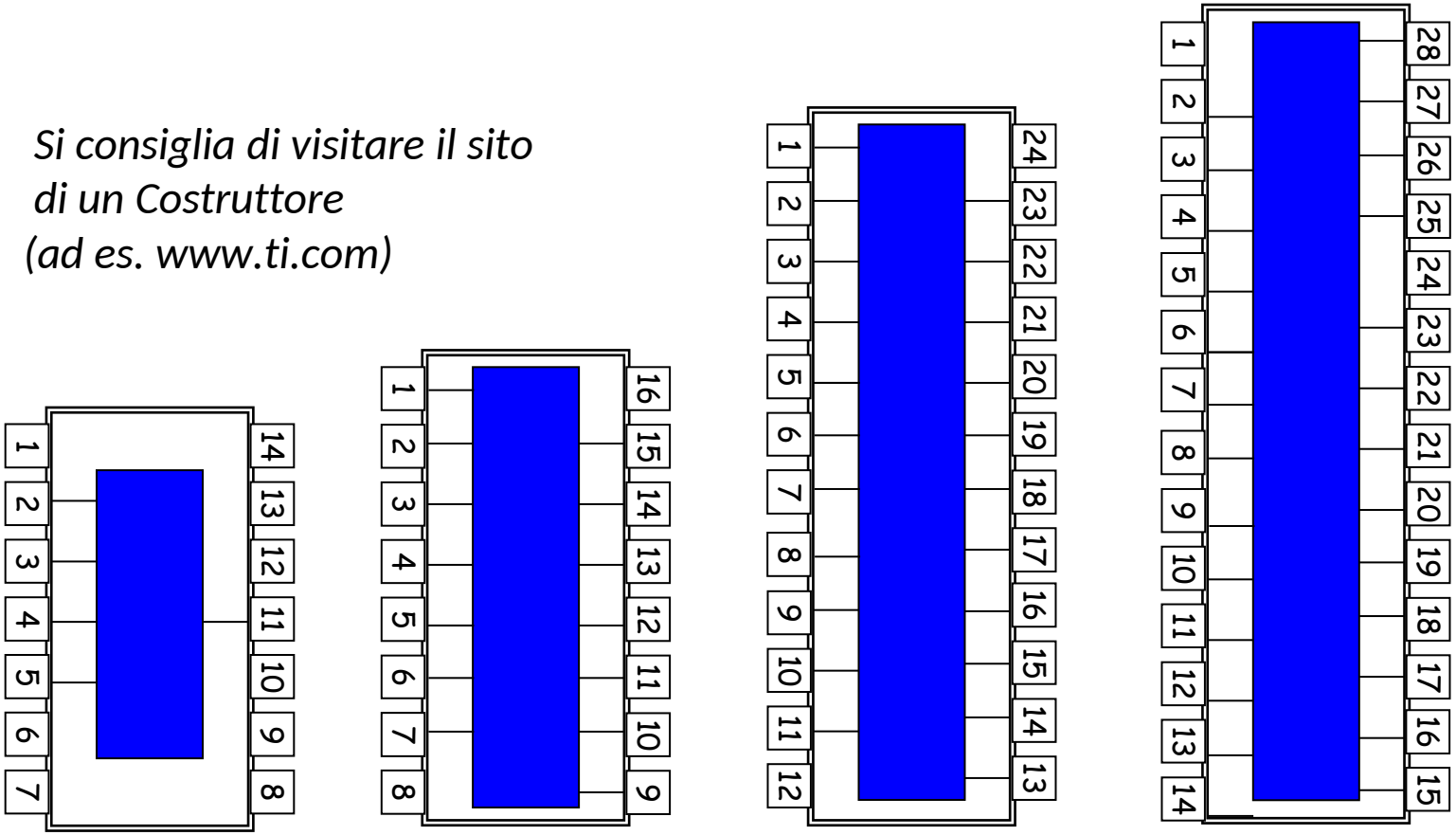
Progetto logico di circuiti combinatori



Circuiti combinatori MSI e LSI

- Sono disponibili nella forma di componenti elementari anche reti più complesse delle semplici «famiglie di gate» viste in precedenza
- Tali reti risultano particolarmente utili per il progettista logico, in quanto rappresentano **i blocchi base per risolvere problemi più complessi**
- Vedremo ora l'utilizzo di alcune di tali reti al fine della sintesi combinatoria

Si consiglia di visitare il sito di un Costruttore (ad es. www.ti.com)



Full adder

Decoder
Multiplexer

Unità
Logico-Aritmetica (ALU)

Contatore
Registro

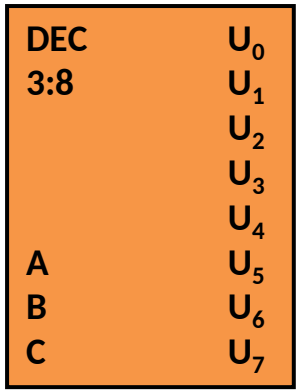
… µP
RAM

Decoder e
sintesi con decoder e OR

Esempio Decoder 3:8

- Rete combinatoria che **converte da numero binario a 3 bit a codice «1 su 8»** (=2^3). Ha quindi 3 ingressi e 8 uscite.
- Una convenzione per indicare gli ingressi usa le prime 3 lettere, con la A ad indicare la cifra **meno significativa**
- Ogni uscita ha solo una configurazione degli ingressi per cui vale «1», quella che **codifica il numero dell'uscita stessa**
- Quindi la sintesi canonica (e anche minima) SP prevede quell'unico mintermine nell'espressione

C	B	A	U_7	U_6	U_5	U_4	U_3	U_2	U_1	U_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



Sintesi canonica/minima SP:

$$U_0 = \overline{C}\,\overline{B}\,\overline{A}$$

$$U_1 = \overline{C}\,\overline{B}\,A$$

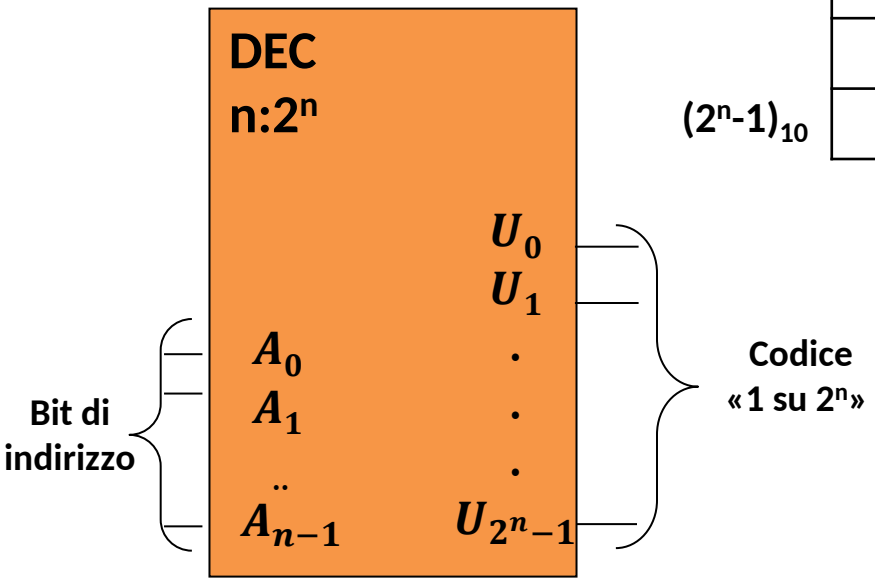
...

$$U_7 = C\,B\,A$$

Il decoder generico DEC n:2^n

- Rete di trascodifica da codice binario a n bit a codice «1 su 2^n»
- Gli n ingressi vengono anche indicati come indirizzi (A, *address*), con A_0 indirizzo di minor peso
- L'indice i dell'uscita U_i attivata è pari al numero rappresentato dalla configurazione binaria degli ingressi: $i = A_{n-1} \cdot 2^{n-1} + \ldots + A_1 \cdot 2^1 + A_0 \cdot 2^0$

	A_{n-1}	..	A_2	A_1	A_0	U_{2^n-1}	..	U_2	U_1	U_0
$(0)_{10}$	0	..	0	0	0	0	..	0	0	1
$(1)_{10}$	0	..	0	0	1	0	..	0	1	0
$(2)_{10}$	0	..	0	1	0	0	..	1	0	0
	0	..	0	1	1	0	..	0	0	0
	..	..	..	..	..	..	..	..	..	..
$(2^n-1)_{10}$	1	..	1	1	1	1	..	0	0	0



Sintesi canonica/minima SP:

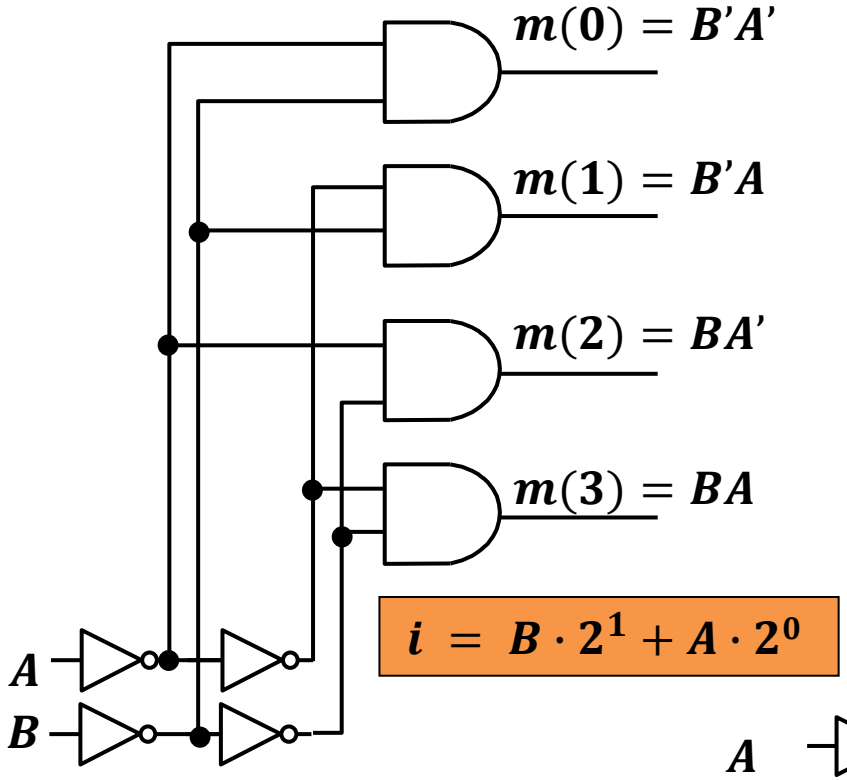
$$U_0 = \overline{A_0}\,\overline{A_1}\,\overline{A_2} \ldots \overline{A_{n-1}}$$

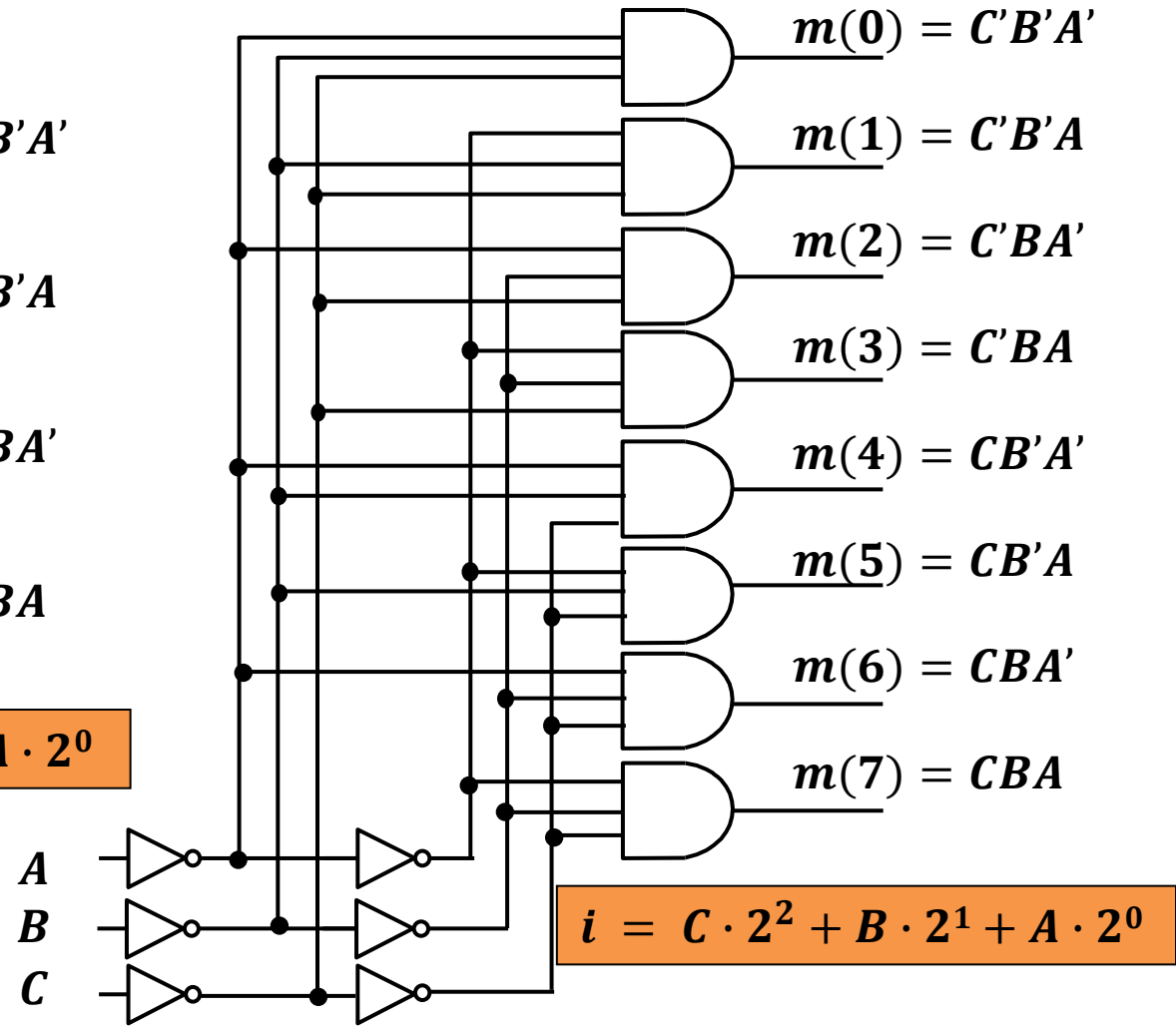
$$U_1 = A_0\,\overline{A_1}\,\overline{A_2} \ldots \overline{A_{n-1}}$$

..

$$U_{2^n-1} = A_0\,A_1\,A_2 \ldots A_{n-1}$$

I DECODER 2:4 e 3:8



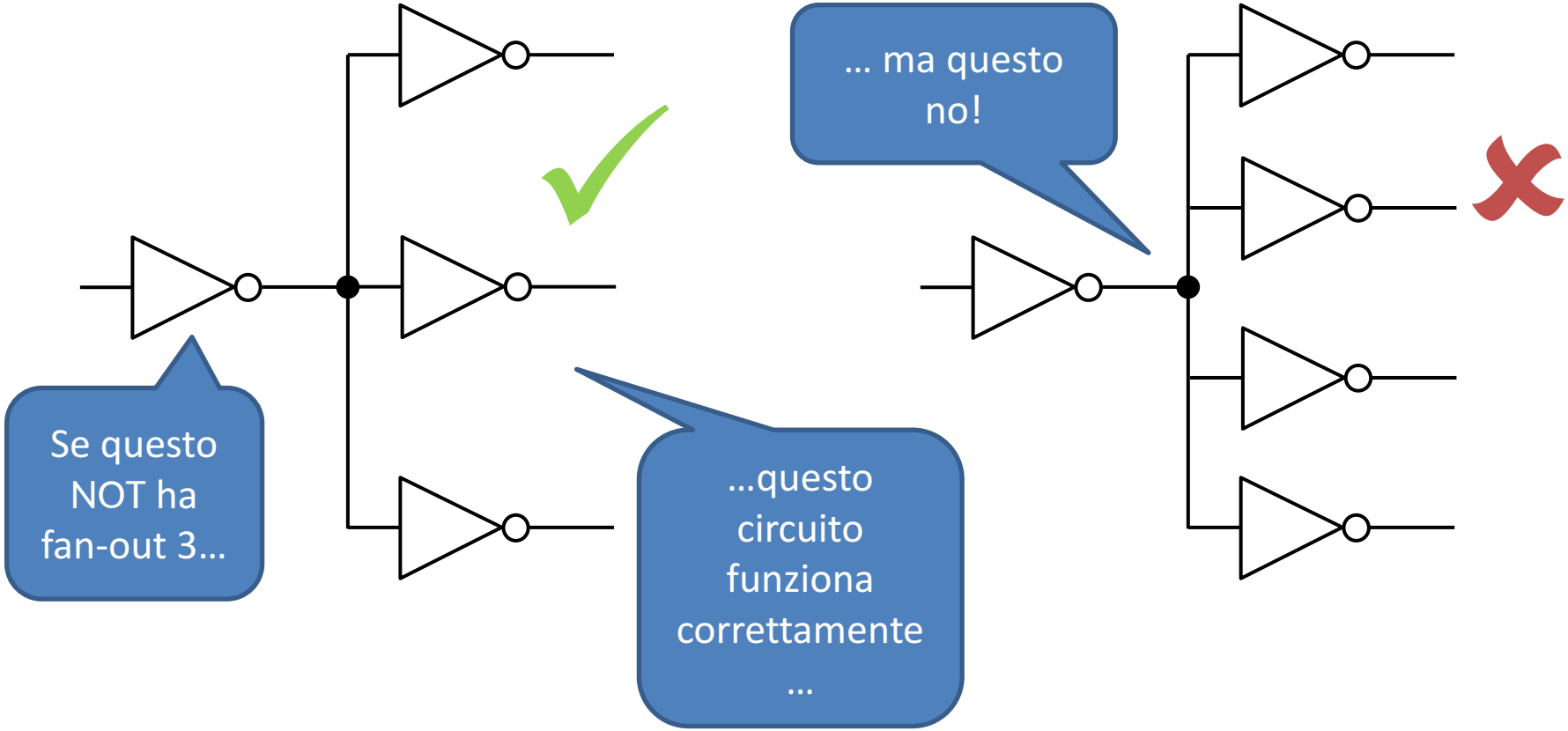


- 2^n AND a n ingressi, $2n$ NOT (e non n per gestire il fan-out, v. prossime slide)
- Il decoder realizza **tutti i possibili mintermini** di una espressione a n variabili
- In questa rappresentazione, A è il bit in ingresso di peso minore

Effetto di carico: Fan-out

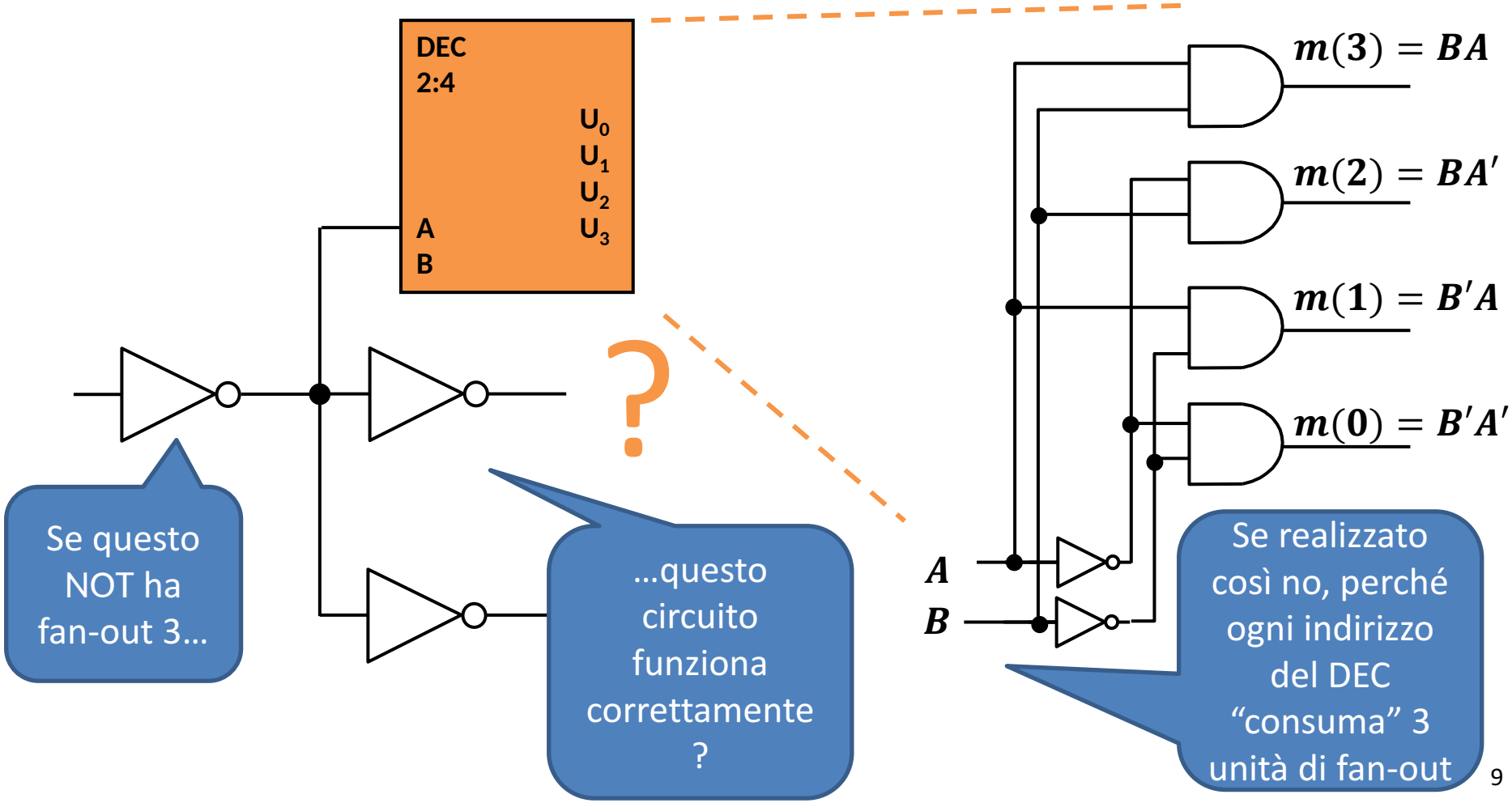
L'uscita di un gate è caratterizzata da un fan-out, ovvero un numero massimo di ingressi di altri gate a cui può essere collegata.

Il **fan-out** di un gate dipende dalla tecnologia con cui è realizzato ed è generalmente alto (>10), ma è comunque limitato



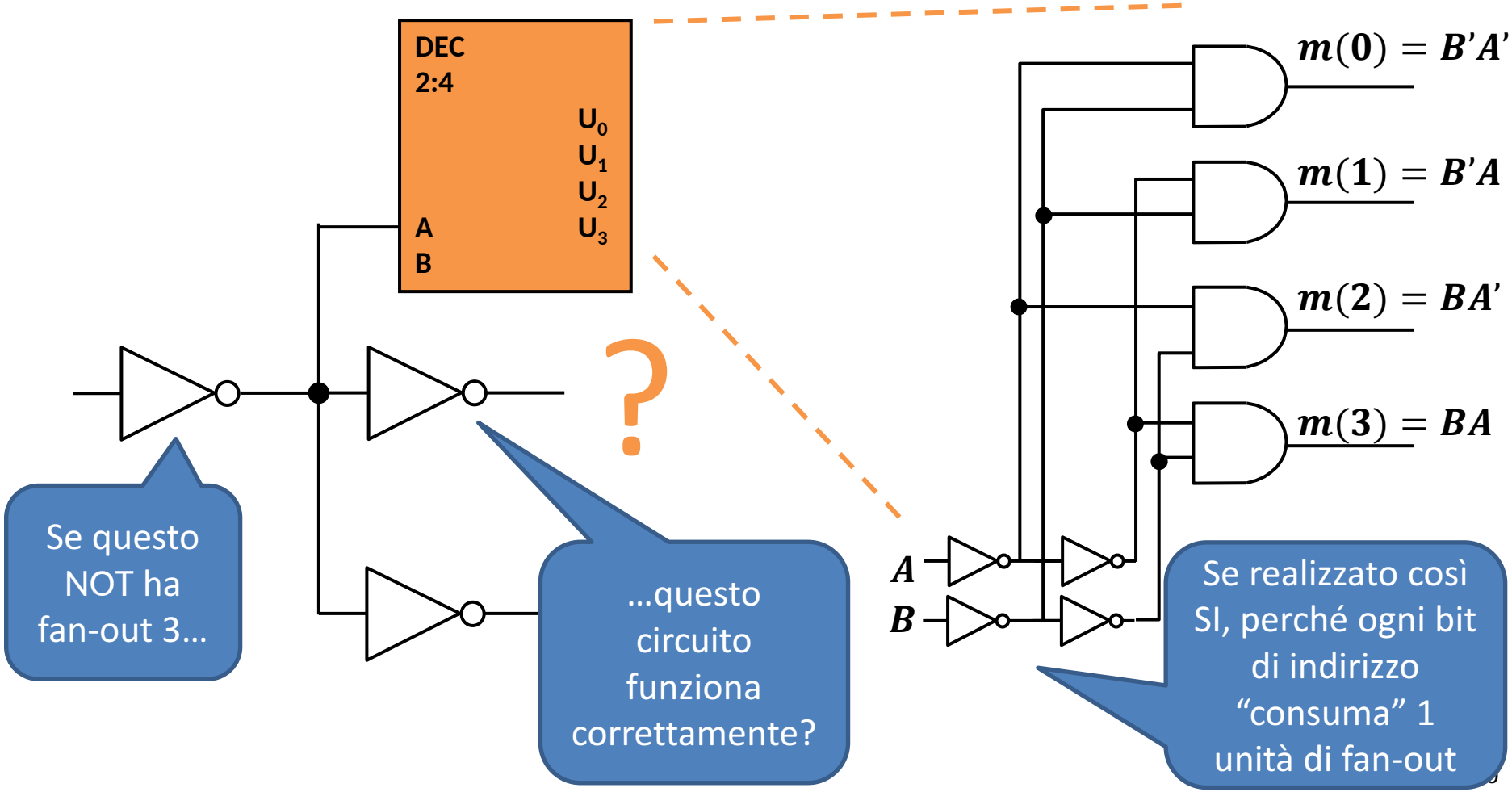
Effetto di carico: Fan-out

Problema: quando connetto l'uscita di un gate ad un componente più complesso, per esempio un DEC 3:8, **non è chiaro** senza conoscere la struttura interna del componente **quanto fan-out «consuma» quel circuito**, e quindi quanto ne «rimane».



Effetto di carico: Fan-out

Soluzione: **ogni ingresso di un circuito complesso deve «consumare» un'unità di fan-out**, ovvero esporre carico unitario all'esterno. Se l'ingresso deve alimentare più di un gate interno al circuito**, possiamo usare due NOT in cascata** per avere il segnale in forma vera e negata senza far consumare più carico all'ingresso.

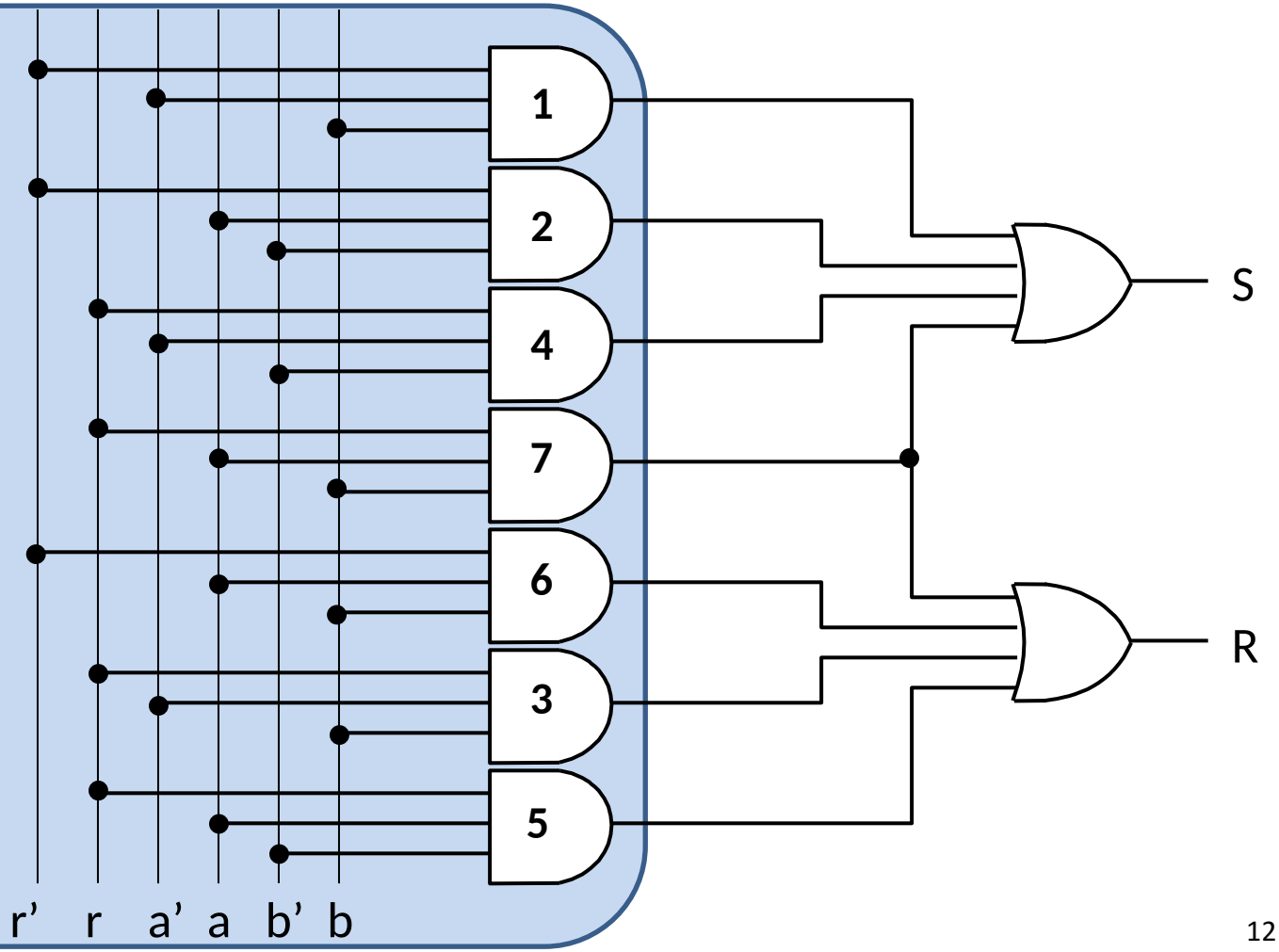


Sintesi canonica SP del Full Adder

$S = \sum_3 m\,(1,2,4,7) = a'\,b'\,r + a'\,b\,r' + a\,b'\,r' + a\,b\,r$

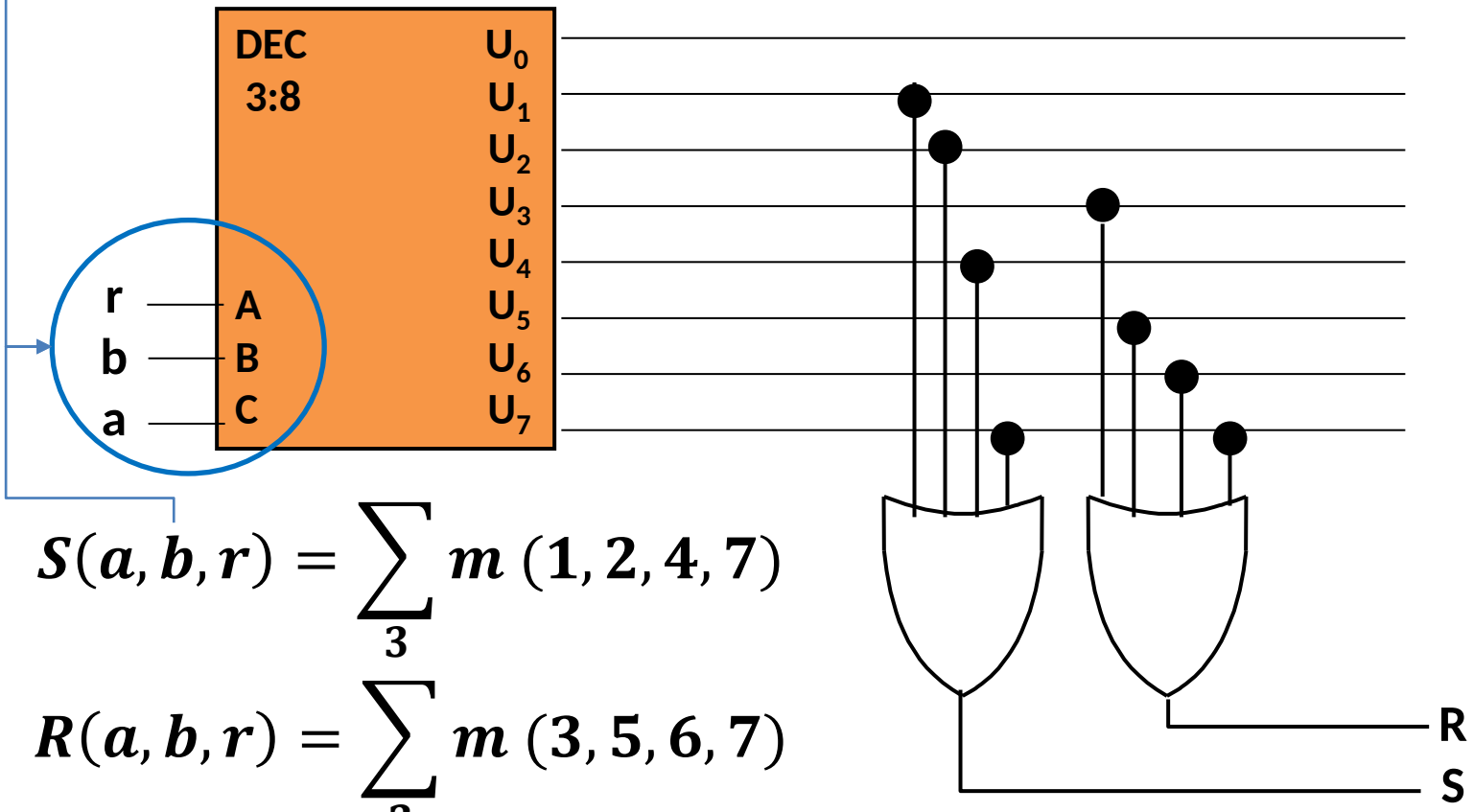
$R = \sum_3 m\,(3,5,6,7) = a'\,b\,r + a\,b'\,r + a\,b\,r' + a\,b\,r$

Segnali negati e mintermini sono già realizzati in un DEC 3:8 ai cui ingressi siano portati i tre segnali a, b, r



Sintesi del Full Adder con Decoder e Or

- Utilizzando opportunamente Decoder e gate OR **si può sintetizzare qualsiasi espressione in forma canonica SP**
- Non servono mappe e non è una sintesi di costo minimo, però tempo di progettazione quasi zero. ATTENZIONE: Devo **collegare gli ingressi ai bit di indirizzo nell'ordine usato per definire i mintermini**.

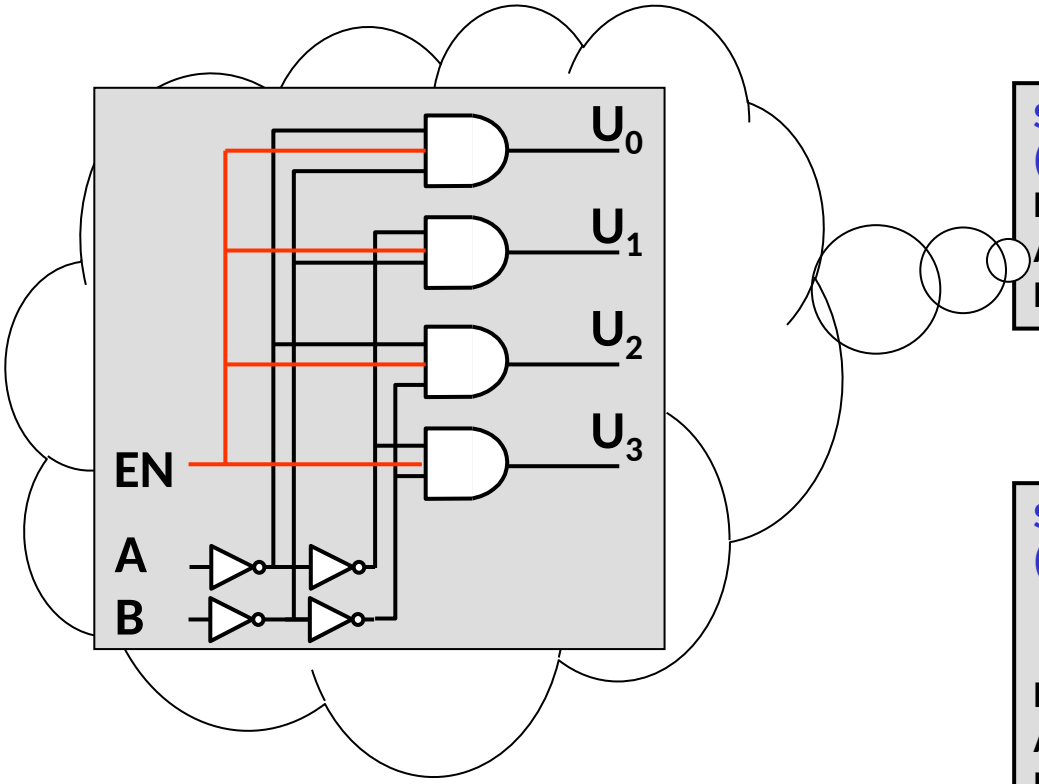


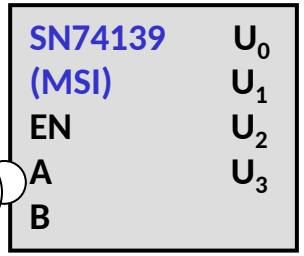
$$S(a,b,r) = \sum_{3} m\,(1,2,4,7)$$

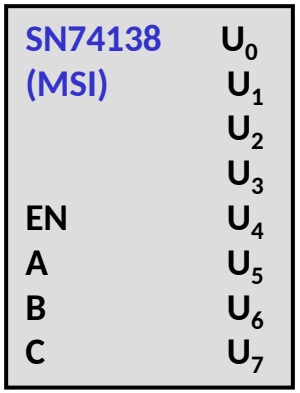
$$R(a,b,r) = \sum_{3} m\,(3,5,6,7)$$

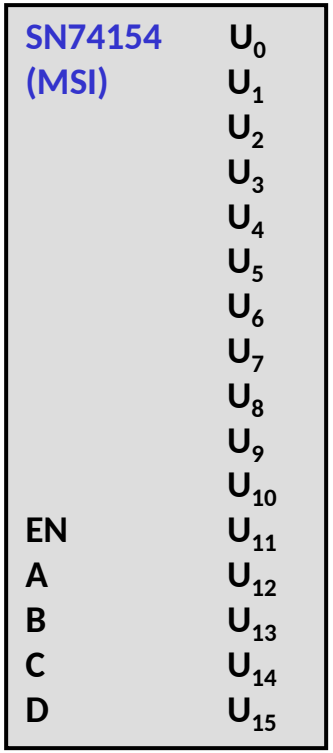
Il circuito integrato DECODER

- **Esistono Decoder** in forma integrata con 2, 3 o 4 bit di indirizzo
- Medium Scale Integration (**MSI**)









Segnale di enable: l'ingresso EN=0 **disabilita tutte le uscite (rappresenta la stringa di «zeri» del codice 1 su N)**

Realizzazione modulare di un Decoder 4:16

- Limite della sintesi con Decoder: con n elevato (es. $n > 5$) i Decoder non sono disponibili, in quanto il numero di componenti e di uscite cresce esponenzialmente con n

- Soluzione: **composizione modulare** (il prodotto logico gode della **proprietà associativa**), per es.

  $U_9 = 1$ se $D\bar{C}\bar{B}A = (D\bar{C})(\bar{B}A) = 1$

  ovvero se $D\ \bar{C} = 1$ e $\bar{B}\ A = 1$

  ovvero se DC rappresentano in binario il numero 2 =$(10)_2$ e BA il numero 1=$(01)_2$

- Quindi posso decodificare i gruppi di bit con due decoder più piccoli 2:4 e comporre i risultati

4 bit
16 conf.

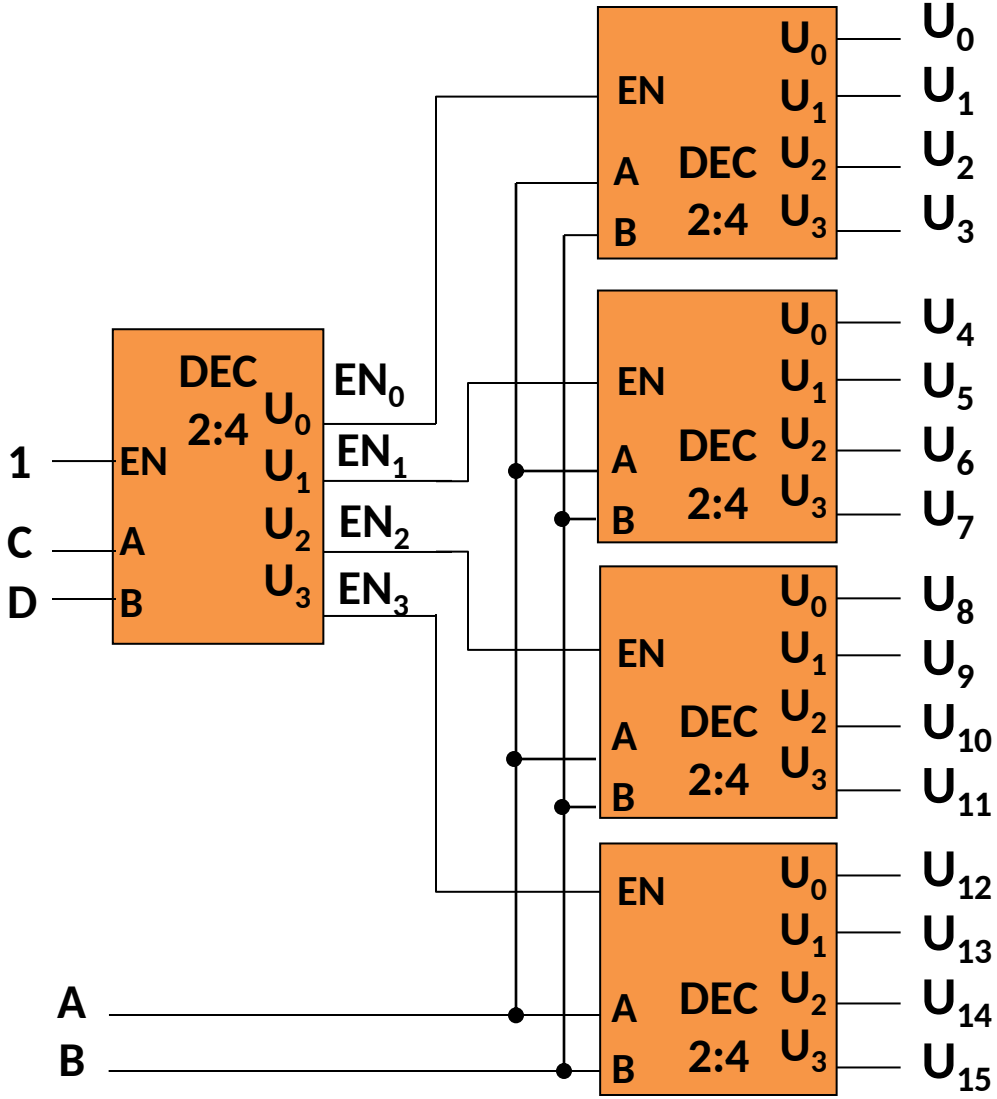
D	C	B	A	$U_i = 1$
0	0	0	0	$i = 0$
0	0	0	1	$i = 1$
0	0	1	0	$i = 2$
0	0	1	1	$i = 3$
0	1	0	0	$i = 4$
0	1	0	1	$i = 5$
0	1	1	0	$i = 6$
0	1	1	1	$i = 7$
1	0	0	0	$i = 8$
1	0	0	1	$i = 9$
1	0	1	0	$i = 10$
1	0	1	1	$i = 11$
1	1	0	0	$i = 12$
1	1	0	1	$i = 13$
1	1	1	0	$i = 14$
1	1	1	1	$i = 15$

Realizzazione modulare di un Decoder 4:16

- Quando decodifico i bit di peso maggiore, genero i segnali di enable per i decoder collegati ai bit di minor peso, così i decoder di secondo livello non interessati dalla decodifica produrranno 0 su tutti i bit di uscita

$$U_9 = EN_2\ \bar{B}\ A\ = (D\ \bar{C})\ \bar{B}\ A$$
$$= D\ \bar{C}\ \bar{B}\ A$$

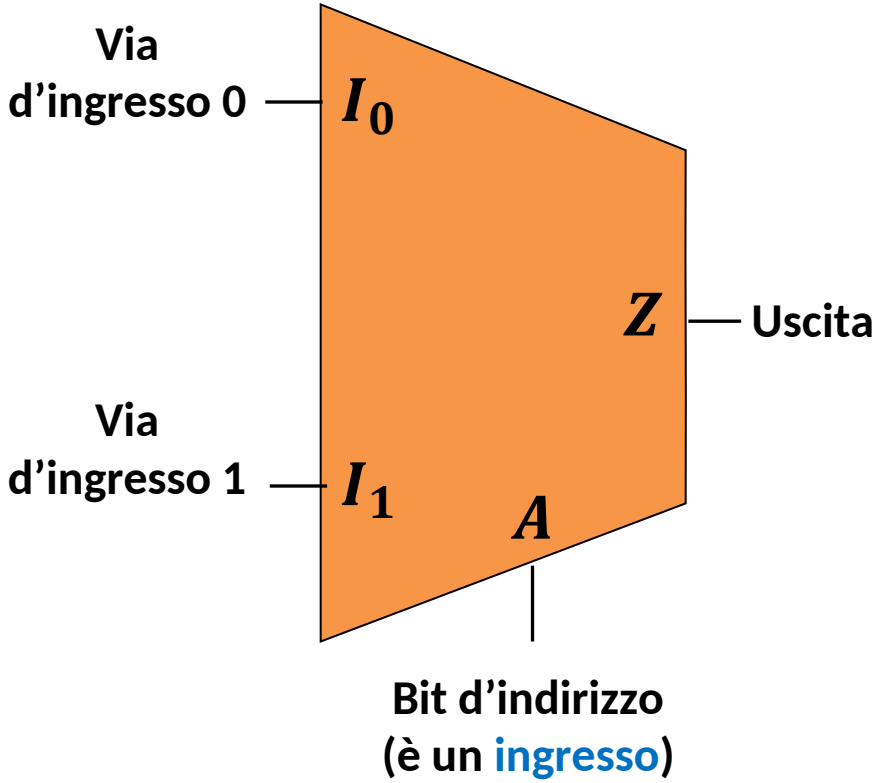
- **IMPORTANTE: non posso connettere gli indirizzi in modo arbitrario. Devo collegare il bit più significativo all'ingresso corrispondente, etc..**
- **Indirizzi di peso minore (bit meno significativi) devono essere portati negli stadi a valle (es. AB)**



Multiplexer

Multiplexer (selettore) a 2 vie

- Un **selettore** è l'equivalente Hardware di un «if»: permette di decidere, tramite un segnale A, quale tra le due vie d'ingresso I_0 I_1 sarà replicata dall'uscita
- Per convenzione, per rappresentare un multiplexer si usa un trapezio invece del solito rettangolo
- La tabella della verità (in forma di mappa di Karnaugh) e l'equazione SP risultante sono le seguenti:



A \ I_1I_0	00	01	11	10
0	0	1	1	0
1	0	0	1	1

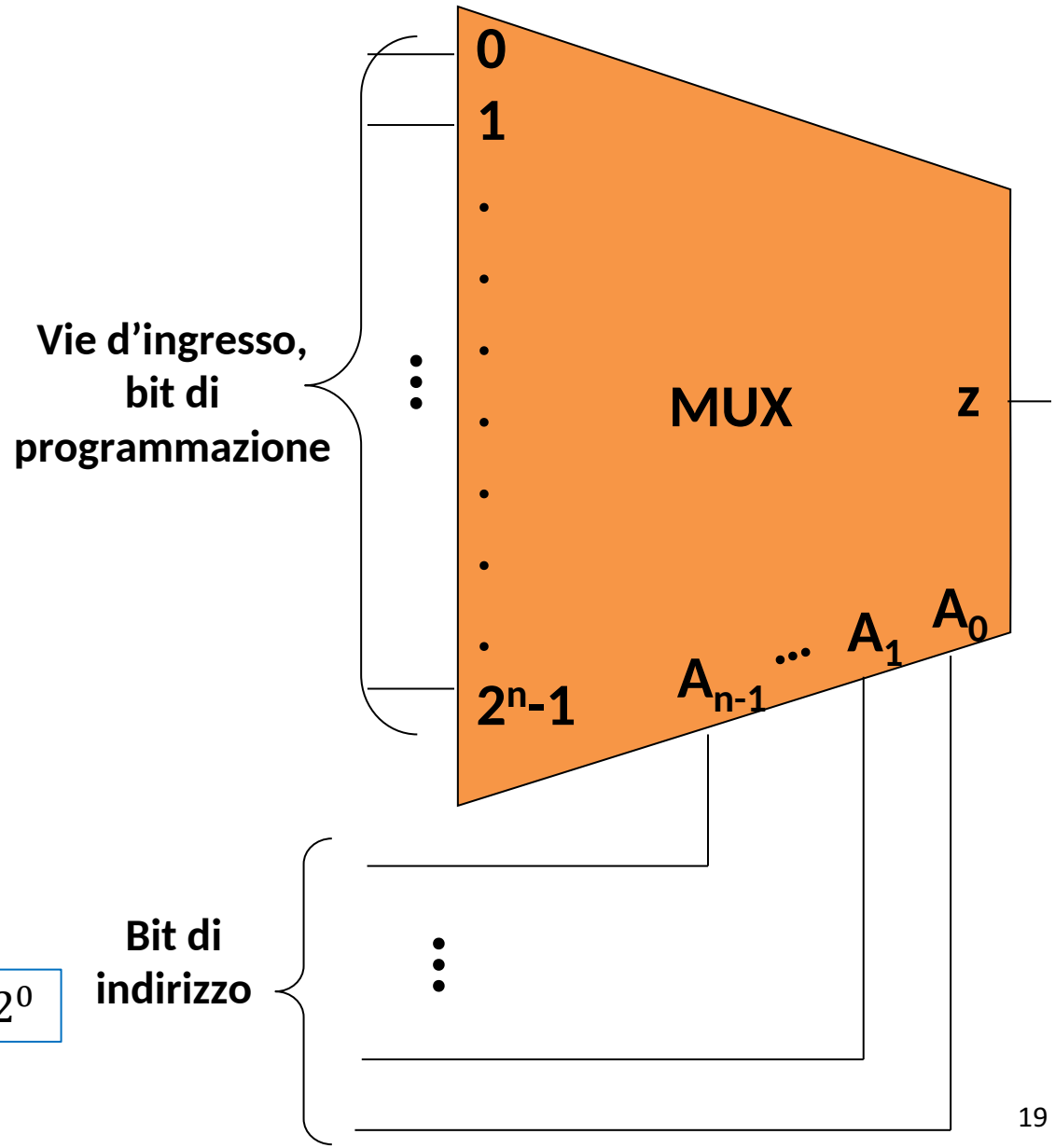
$$Z = A'I_0 + AI_1$$

if $A =$ «1» then $Z = I_1$
else $Z = I_0$

Multiplexer (MUX) generico

- Il concetto di selettore può essere generalizzato a n bit di indirizzo
- Gli n bit di indirizzo selezionano uno tra i 2^n ingressi detti «vie» o anche «bit di programmazione» a seconda dell'uso che si fa del MUX (come **selettore** o **generatore di funzioni**, v. prossime slide)
- Al crescere di n **cresce esponenzialmente** il numero delle vie
- L'ingresso replicato sull'uscita si determina dai bit di indirizzo secondo la stessa relazione vista per il DEC.

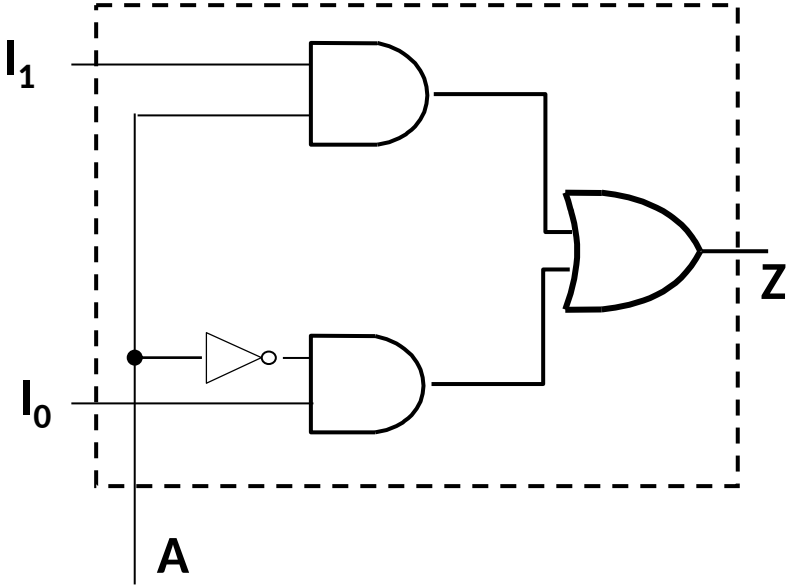
$$i = A_{n-1}\,2^{n-1} + \ldots + A_1 2^1 + A_0\,2^0$$



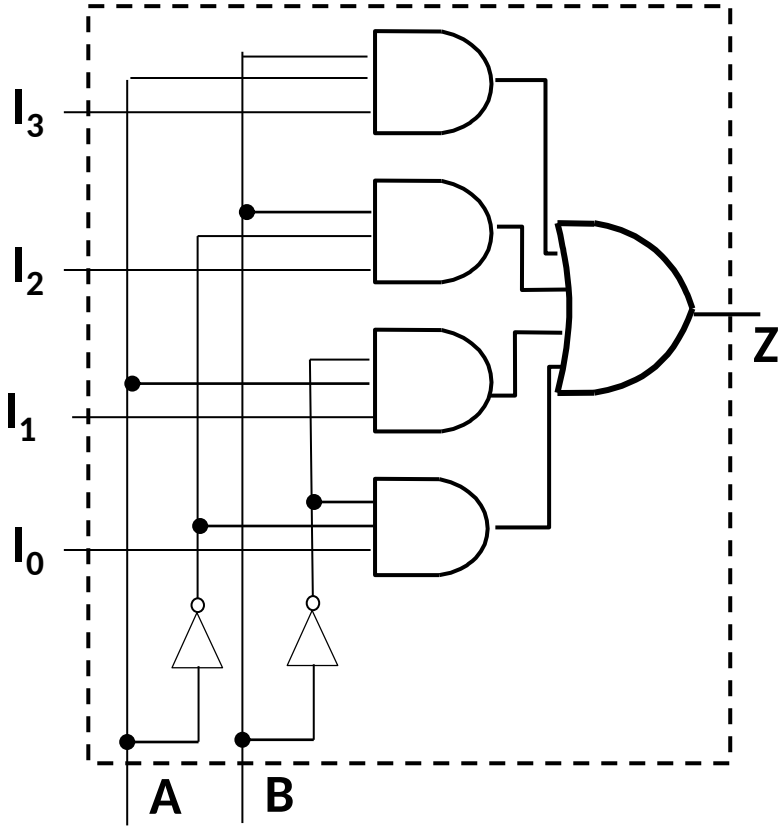
Multiplexer a 4 vie

- La sua struttura può essere dedotta generalizzando quella di un MUX a 2 vie
- 2 vie: 2 AND a 2 ingressi (uno per la via e 1 per l'indirizzo) e un OR a 2 ingressi
- 4 vie: 4 AND a 3 ingressi (2 bit di indirizzo + 1 via di ingresso) e 1 OR a 4ingressi
- 2 (o meglio 4 come nel decoder) NOT per avere in forma vera e complementata i 4 bit di indirizzo

$$Z = A'I_0 + AI_1$$



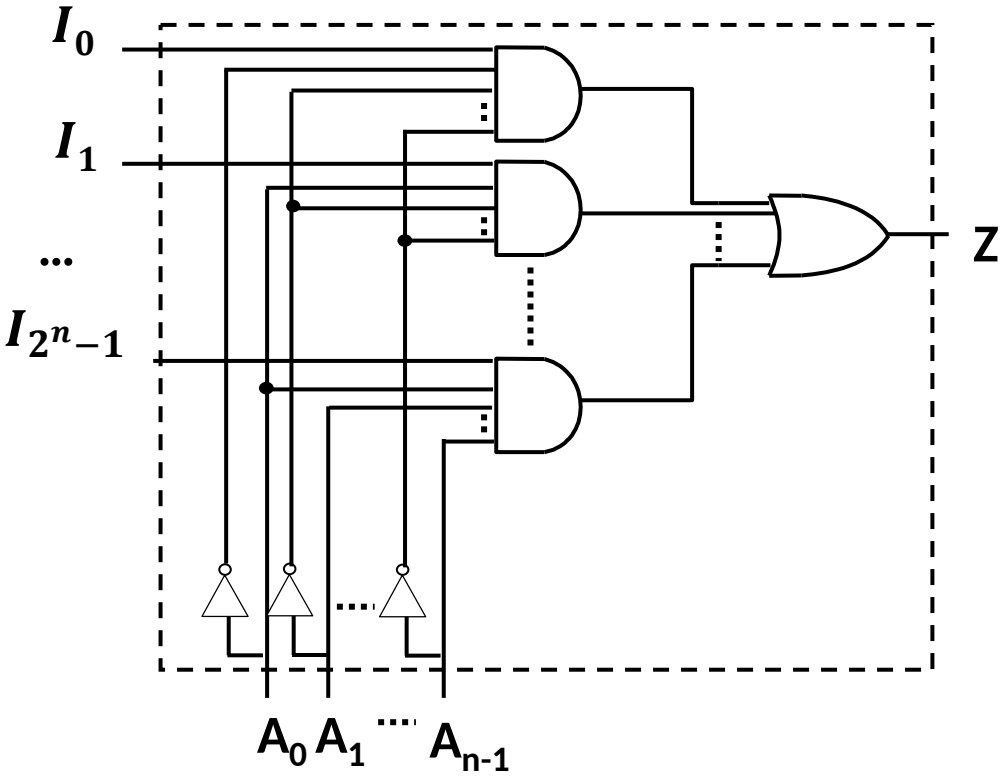
2 vie, 1 bit di indirizzo



4 vie, 2 bit di indirizzo

Multiplexer a 2^n vie

- La sua struttura può essere dedotta generalizzando quella di un MUX a 4 vie
- 4 vie: 4 AND a 3 ingressi (due per la via e 1 per l'indirizzo) e un OR a 4 ingressi
- 2^n vie: 2^n AND a $n+1$ ingressi (n bit di indirizzo + 1 via di ingresso) e 1 OR a 2^n ingressi
- n (o meglio $2n$ come nel decoder) NOT per avere in forma vera e complementata gli n bit di indirizzo



2^n vie, n bit di indirizzo

Teorema di espansione di Shannon

- In realtà enunciato da Boole
- Data una funzione F di n variabili binarie, vale la relazione

$$F(x_1, \dots, x_i, \dots, x_n) =$$
$$x_i\, F(x_1, \dots, 1, \dots x_n) + \overline{x_i}\, F(x_1, \dots, 0, \dots x_n)$$

- Dimostrazione: Per le equivalenze E5 e E6 (limite e identità)
  - Quando $x_i = 1$, entrambi i lati sono uguali a $F(x_1, \dots, 1, \dots x_n)$
  - Quando $x_i = 0$, entrambi i lati sono uguali a $F(x_1, \dots, 0, \dots x_n)$

- E' valida anche la relazione duale

$$F(x_1, \dots, x_i, \dots, x_n) =$$
$$\big(x_i + F(x_1, \dots, 0, \dots x_n)\big)\,\big(\overline{x_i} + F(x_1, \dots, 1, \dots x_n)\big)$$

Espressioni generali

- Applicando tre volte il teorema di espansione ad una funzione di 3 variabili, otteniamo

$$F(x_1, x_2, x_3) =$$

$$\overline{x_1} F(0, x_2, x_3) + x_1\, F(1, x_2, x_3) =$$

$$\overline{x_1}\,\overline{x_2}\, F(0,0, x_3) + \overline{x_1}\, x_2\, F(0,1, x_3) + x_1\, \overline{x_2}\, F(1,0, x_3) + x_1\, x_2\, F(1,1, x_3) =$$

Valore di F in 0

Mintermine 0

$$\overline{x_1}\,\overline{x_2}\,\overline{x_3}\, F(0,0,0) + \overline{x_1}\,\overline{x_2}\, x_3\, F(0,0,1) + \overline{x_1}\, x_2\, \overline{x_3}\, F(0,1,0) + \overline{x_1}\, x_2\, x_3\, F(0,1,1) + x_1 \overline{x_2}\,\overline{x_3} F(1,0,0) + x_1 \overline{x_2}\, x_3 F(1,0,1) + x_1\, x_2 \overline{x_3}\, F(1,1,0) + x_1\, x_2\, x_3\, F(1,1,1)$$

Mintermine 6

Valore di F in 6

Espressioni generali

In generale, applicando ripetutamente il **teorema di espansione di Shannon** è possibile dedurre le seguenti **espressioni generali**

- **<u>Caso SP:</u>** ogni funzione di n variabili è descritta da una espressione in cui compaiono, in somma logica, tutti i mintermini di n variabili ciascuno in prodotto logico con il valore della funzione quando in ingresso compare la configurazione riconosciuta dal mintermine

$$\boldsymbol{F(x_1, x_2, \dots, x_n) = \sum_{i=0}^{2^n-1} m(i) \cdot F(i)}$$

$\boldsymbol{m(i)}$: i-simo mintermine di n bit
$\boldsymbol{F(i)}$: **valore** della funzione relativo alla configurazione per cui $m(i) = 1$

- **<u>Caso PS:</u>** ogni funzione di n variabili è descritta da una espressione in cui compaiono, in prodotto logico, tutti i maxtermini di n variabili ciascuno in somma logica con il valore della funzione quando in ingresso compare la configurazione riconosciuta dal maxtermine

$$\boldsymbol{F(x_1, x_2, \dots, x_n) = \prod_{i=0}^{2^n-1} (M(i) + F(i))}$$

$\boldsymbol{M(i)}$: i-simo maxtermine di n bit
$\boldsymbol{F(i)}$: **valore** della funzione relativo alla configurazione per cui $M(i) = 0$

Full Adder : espressioni generali

- Partendo dall'espressione canonica SP della funzione R di un Full-Adder, «complichiamola» utilizzando le equivalenze notevoli, per dimostrare che è equivalente alle espressioni generali:

$$R = m(3) + m(5) + m(6) + m(7)$$

$$= a' b r + a b' r + a b r' + a b r$$

Identità – E5

$$R = m(3) \cdot 1 + m(5) \cdot 1 + m(6) \cdot 1 + m(7) \cdot 1$$

$$= a' b r \cdot 1 + a b' r \cdot 1 + a b r' \cdot 1 + a b r \cdot 1$$

Limite – E6 & Identità – E5

$$R = m(3) \cdot 1 + m(5) \cdot 1 + m(6) \cdot 1 + m(7) \cdot 1 +$$

$$m(0) \cdot 0 + m(1) \cdot 0 + m(2) \cdot 0 + m(4) \cdot 0$$

$$= a' b r \cdot 1 + a b' r \cdot 1 + a b r' \cdot 1 + a b r \cdot 1 +$$

$$a' b' r' \cdot 0 + a' b' r \cdot 0 + a' b r' \cdot 0 + a b' r' \cdot 0$$

Il MUX come rete programmabile

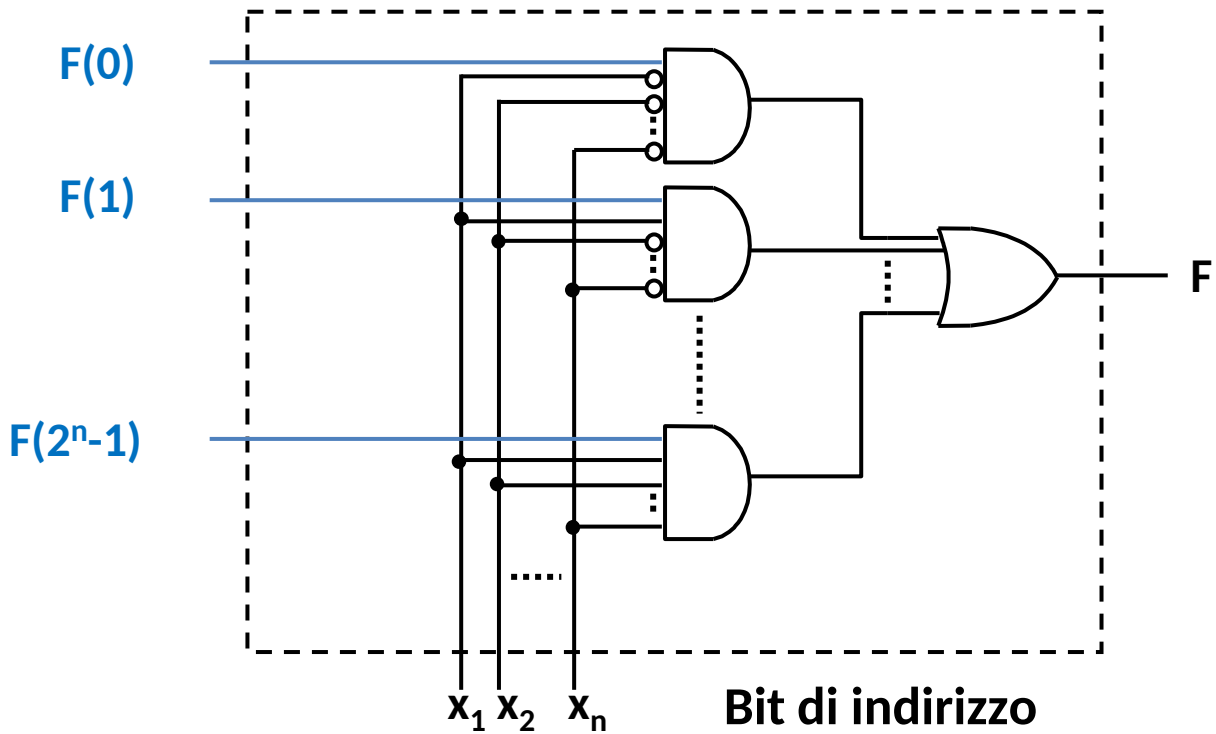
- Il MUX si adatta bene a realizzare l'espressione generale nel caso SP
- $\boldsymbol{n}$ **variabili** -> occorre un MUX a $\boldsymbol{n}$ **bit di indirizzo**
- il MUX viene qui utilizzato come ***generatore di funzioni*** (componente ***programmabile*** a seconda della funzione che si vuole realizzare)

$$F(x_1, x_2, \dots, x_n) = \sum_{i=0}^{2^n-1} m(i) \cdot F(i)$$

Espressione generale SP

F(i): bit di programmazione

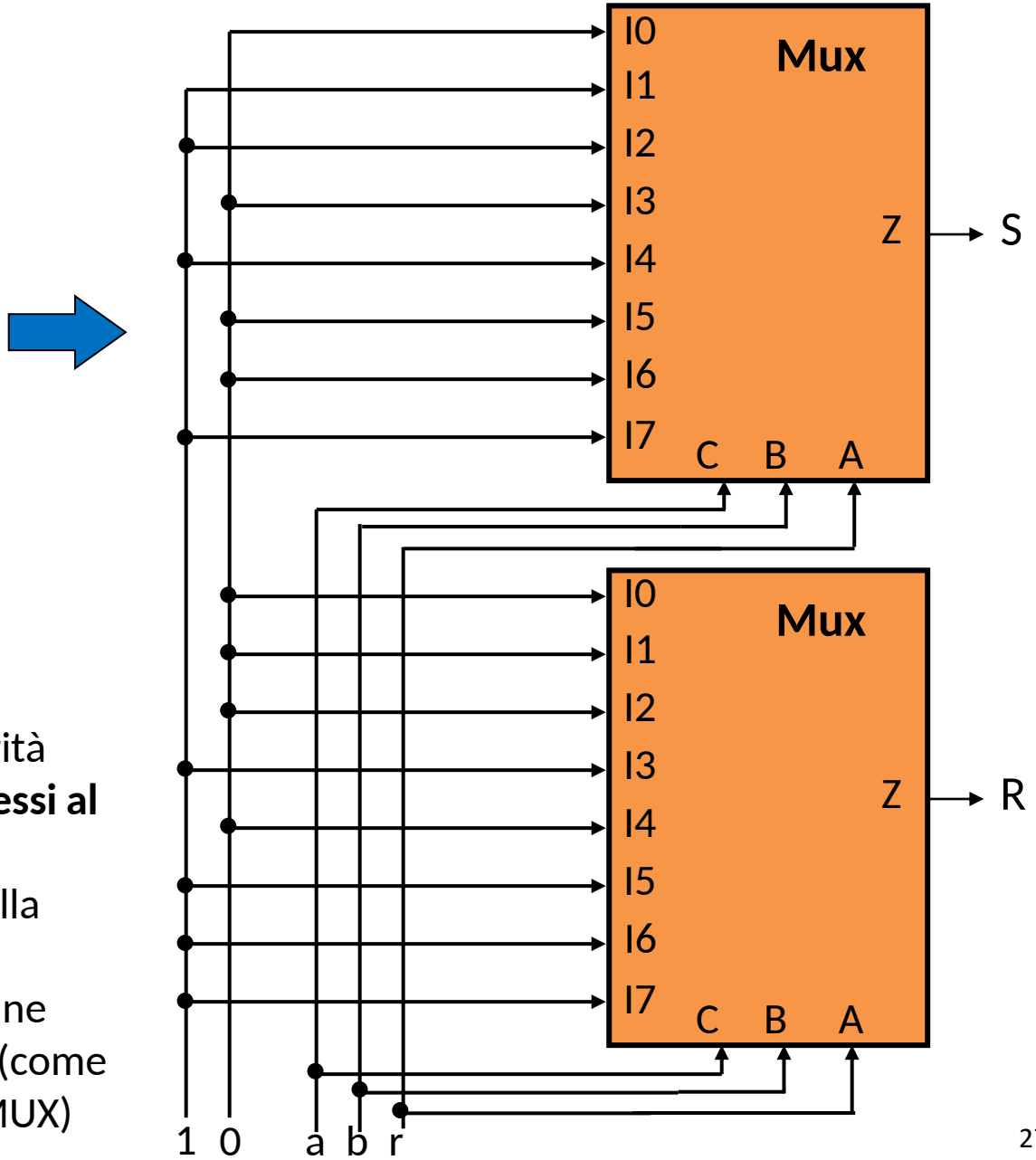
MUX



Quando usato in questo modo, il MUX viene anche detto **LUT** (Look-up Table): basta fornire la TdV sulle vie per sintetizzare la funzione

Sintesi di un full-adder con MUX

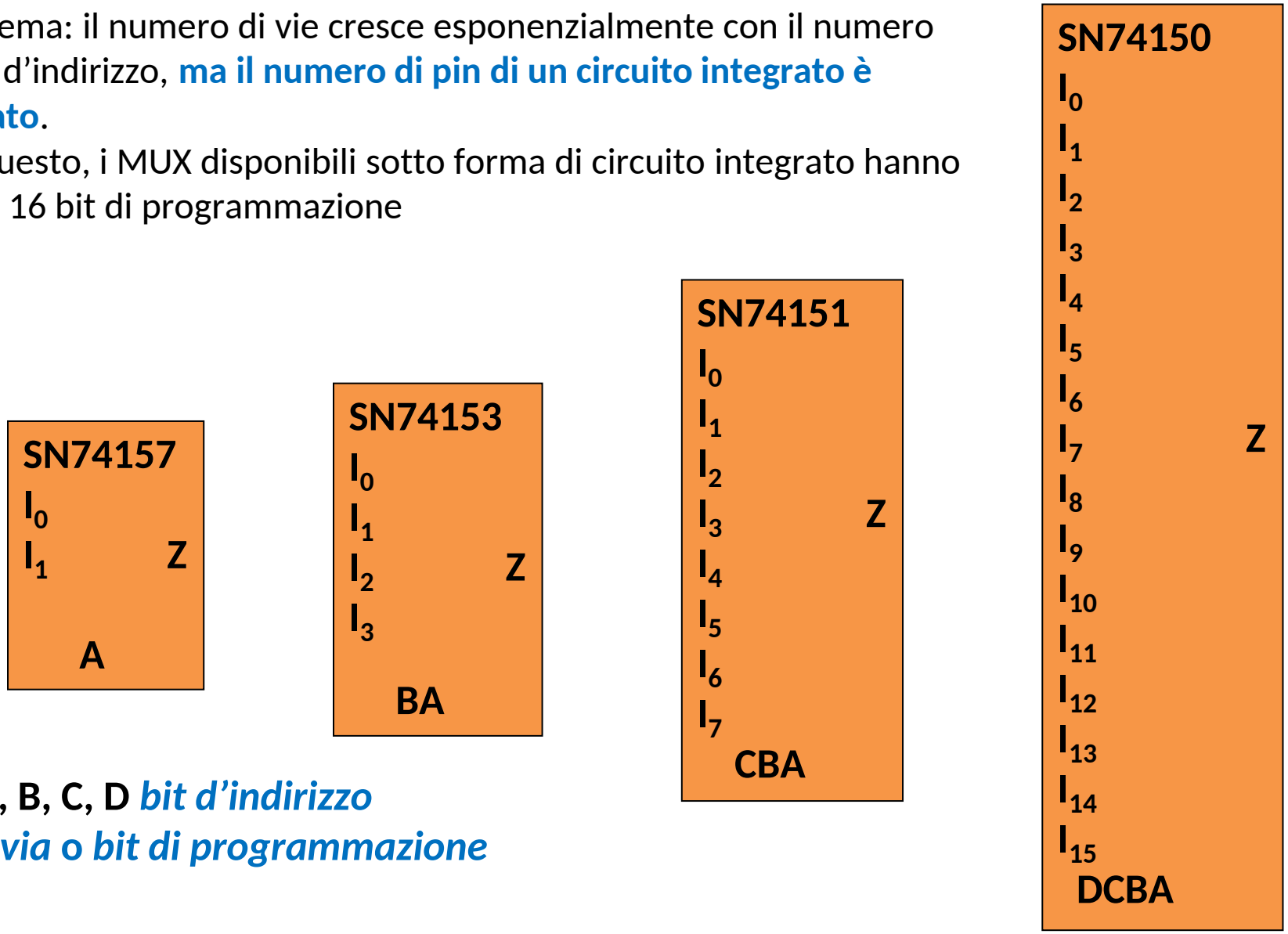
a	b	r	S	R
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



- Riproduco in ingresso al MUX i contenuti della tabella della verità
- **ATTENZIONE: collegare gli ingressi al bit di indirizzo giusto**
- Massimizzo ancora il criterio della **rapidità di progetto**
- La velocità di elaborazione rimane invece quella delle reti minime (come si vede dalla realizzazione del MUX)

I Multiplexer in forma integrata

- Ulteriore esempio di MSI: MUX a 2,4,8 e 16 bit di programmazione
- Problema: il numero di vie cresce esponenzialmente con il numero di bit d'indirizzo, **ma il numero di pin di un circuito integrato è limitato**.
- Per questo, i MUX disponibili sotto forma di circuito integrato hanno al più 16 bit di programmazione

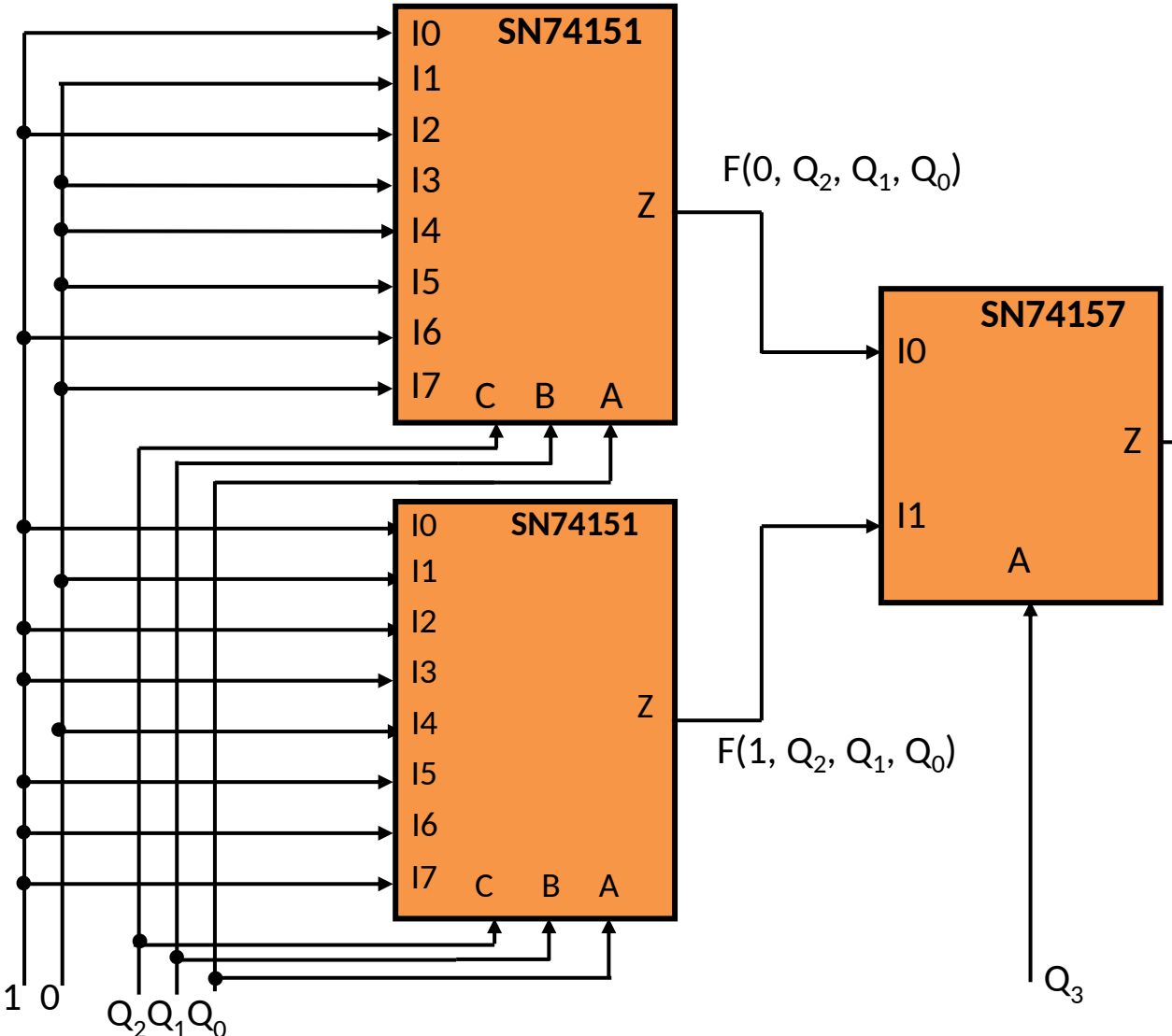


A, B, C, D ***bit d'indirizzo***

I_i ***via o bit di programmazione***

Sintesi a MUX di funzioni di 4 variabili

- La composizione è possibile anche con MUX, **usando il teorema di espansione**
- Per rendere coerenti i bit di programmazione con la tabella della verità, è consigliabile far corrispondere gli indirizzi dei dispositivi **a valle** ai bit di **maggior** peso (es. Q_3)



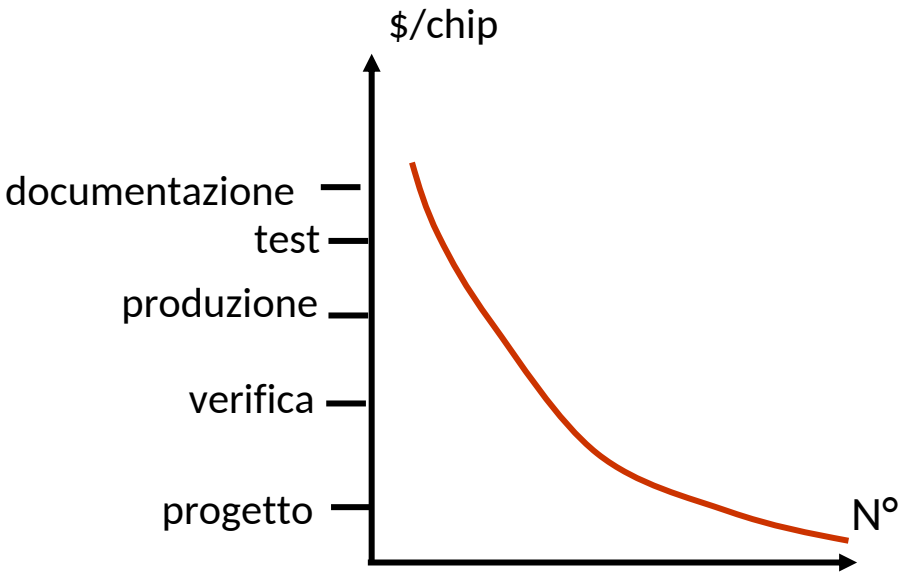
Q_3	Q_2	Q_1	Q_0	Z
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
…	…	…	…	…

…	…	…	…	…
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

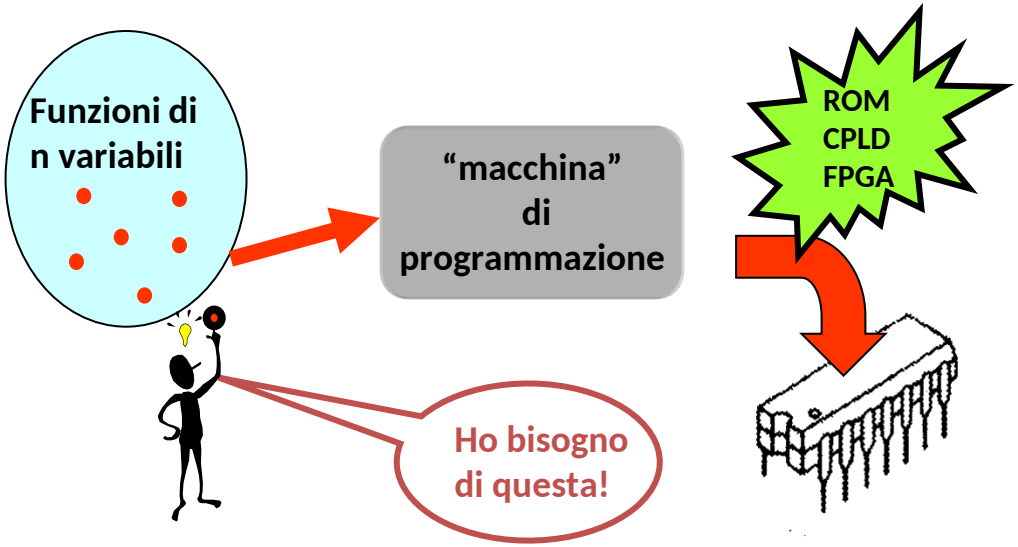
Reti programmabili

VLSI

- Large Scale Integration (**LSI**) e Very Large Scale Integration (**VLSI**):
  - milioni di gate integrati all'interno dello stesso chip
  - comporta costi molto onerosi in tutte le sue fasi di realizzazione
  - per ammortizzarne le spese ne occorre un vasto impiego



- Impossibile la realizzazione ad hoc, si rende necessaria la possibilità di **programmazione**
- Questo permette di ammortizzare i costi elevati attraverso grandi volumi di vendita dello stesso prodotto



Le reti (combinatorie) programmabili

Rete (combinatoria) programmabile: rete (combinatoria) in grado di presentare diverse relazioni ingresso/uscita selezionabili mediante l'attribuzione di una determinata configurazione di valori ad un gruppo di **segnali <u>interni</u>** detti **bit di programmazione**.

N.B.: il MUX può essere visto come rete programmabile, ma con segnali <u>esterni</u>

$$z_i = F(p_1, p_2, \dots, p_y, x_1, x_2, \dots, x_n) = F_p(x_1, x_2, \dots, x_n)$$

TdV come «memoria» non modificabile

- Le reti programmabili possono essere usate per **realizzare reti combinatorie a partire da memorie immutabili** (non-volatili)
- **Una memoria** (per esempio la RAM) **è un circuito che realizza fisicamente un array**, ovvero una lista di dati indicizzabile con un intero (**indirizzo**).
- Una tabella della verità può essere interpretata anche come una **«memoria» non modificabile,** ovvero una lista di 2^n dati immutabili, tra cui gli n ingressi mi permettono di scegliere.
- Quando interrogo la memoria con un **indirizzo** (ovvero presento una configurazione degli ingressi in input alla rete, per esempio **100**) voglio ottenere il **dato** (ovvero le uscite, per esempio **01010010**) scritto in tabella a quella riga.

100

C	B	A	U_7	U_6	U_5	U_4	U_3	U_2	U_1	U_0
0	0	0	0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0	0	1	0
0	1	0	0	1	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	1	0	1	0	0	1	0
1	0	1	0	0	1	0	0	1	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	1	0	0	0	0

01010010

Memorie a sola lettura (ROM)

- Una **Read Only Memory** è proprio un circuito che realizza una memoria a sola lettura, ovvero che contiene ad ogni indirizzo un dato fissato.
- Per quanto detto nella slide precedente, può però essere interpretata anche come **circuito generatore di funzioni combinatorie**
- Es.: ROM con 2 bit di indirizzo

x_1	x_0	$F[7..0]$
0	0	0xAE=1010 1110
0	1	0x94=1001 0100
1	0	0x1D=0001 1101
1	1	0x00=0000 0000

x_1	x_0	F_7	.....	F_0
0	0	1		0
0	1	1	.....	0
1	0	0		1
1	1	0		0

Memorizza 4 byte, uno per ciascuna «cella» identificata dai 2 bit di indirizzo

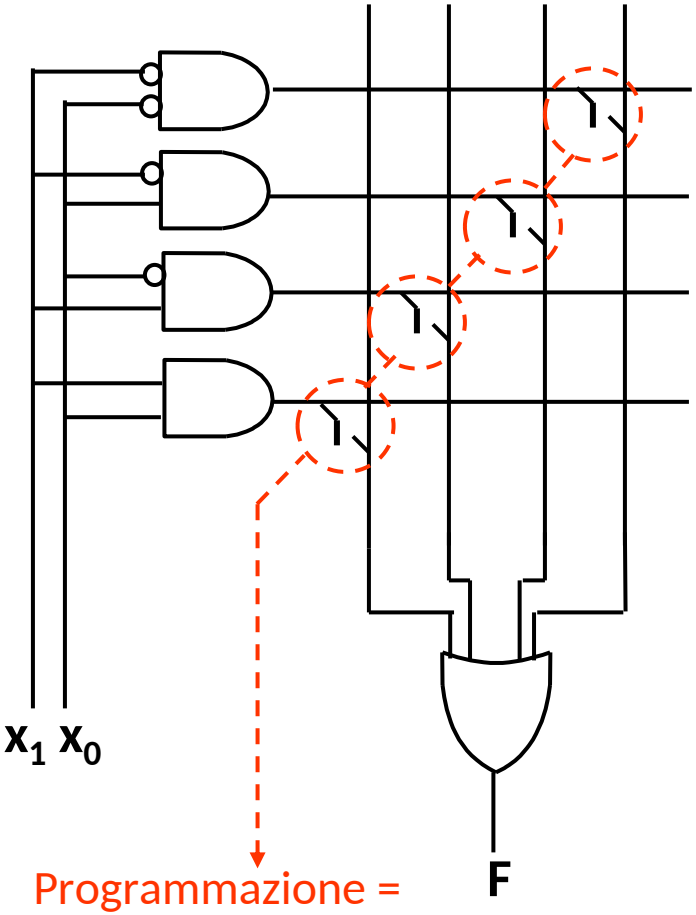
Ma posso vederla anche come 8 *funzioni* $F_0, F_1, \ldots, F_7$ di due ingressi $x_1\ x_0$

- N.B. quando considero le ROM come memorie, raggruppo mentalmente i bit memorizzati per «riga», quando come reti programmabili per «colonna»

Memorie a sola lettura (ROM)

- Interpretare le ROM come generatori di funzioni aiuta anche a comprenderne la realizzazione
- Ogni uscita è realizzata concettualmente tramite la **sintesi Decoder+OR**, ovvero espressioni di tipo SP (2^n AND + 1 OR a elevato fan-in)
- Sono dunque **reti programmabili** in grado di realizzare qualsiasi funzione di n variabili

- Programmabili mediante predisposizione opportuna dei contatti tra la batteria di AND e l'OR e **NON attraverso segnali esterni**
- Perché? Con n=16 ingressi, il contenitore del chip dovrebbe avere 65536 pin per scegliere i 65536 valori di uscita della funzione
- **Si supera così un limite della sintesi a DEC o a MUX**

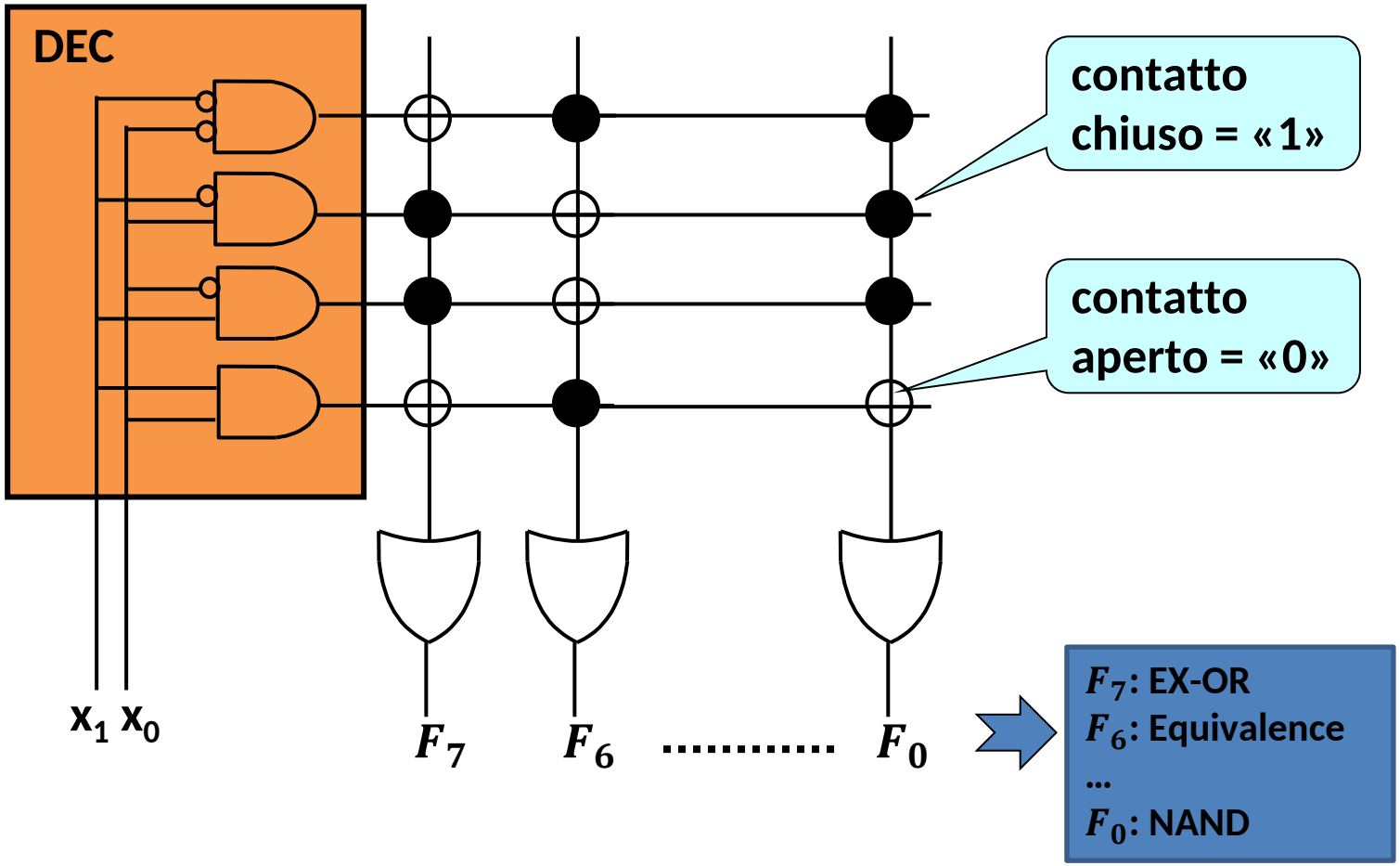


Programmazione = Contatto aperto/chiuso

Il contatto è chiuso se corrisponde a un AND che realizza un mintermine nell'espressione canonica di **F**

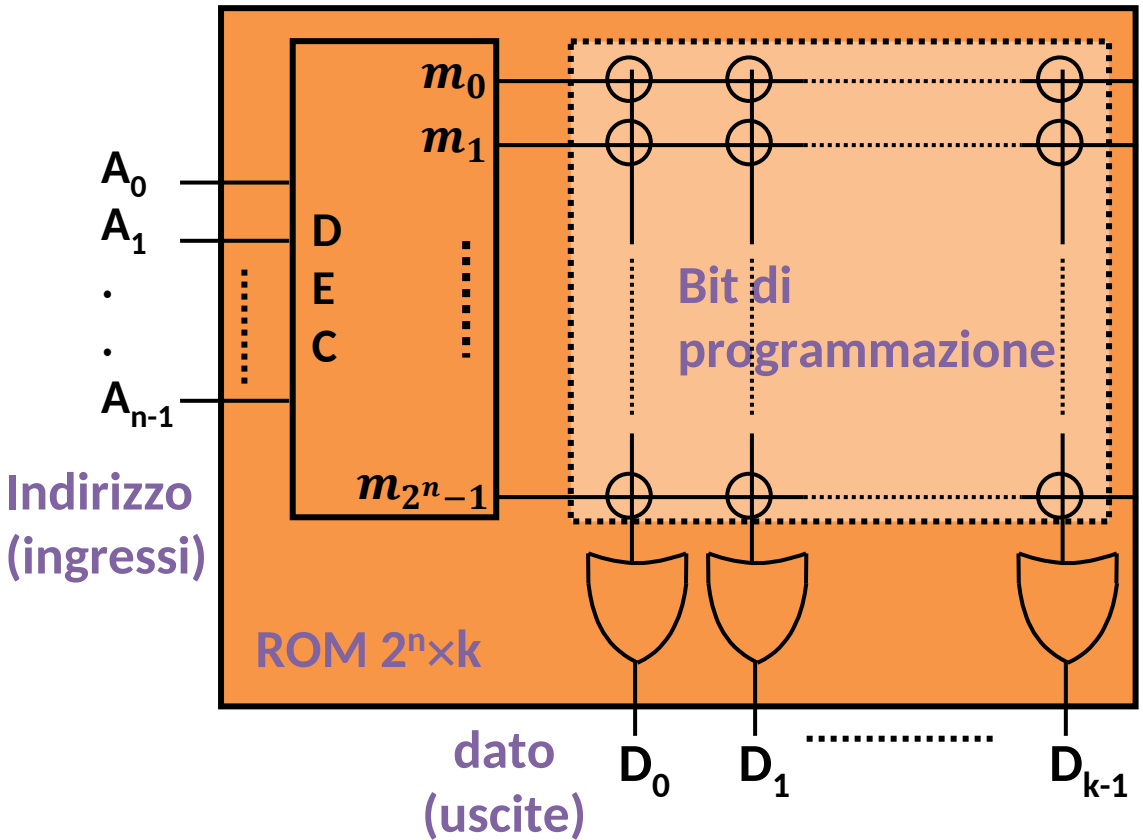
Memorie a sola lettura (ROM)

- Rappresentazione “compatta” della struttura di una ROM :



Le ROM come circuiti di memoria

- Chiudere un contatto equivale dunque a **memorizzare** un «1» piuttosto che uno «0»
  - possiamo interpretare i bit di ingresso come <u>bit d'indirizzo:</u> $A_0, \dots, A_{n-1}$
  - e i bit di uscita come <u>bit di dato:</u> $D_0, \dots, D_{k-1}$
- ROM come circuito di **memoria** in grado di memorizzare $\mathbf{2^n}$ dati, ciascuno di k bit
- Una ROM in grado di memorizzare N dati (ciascuno a k bit) sarà dunque indirizzabile mediante $n = \log_2 N$ bit d'indirizzo



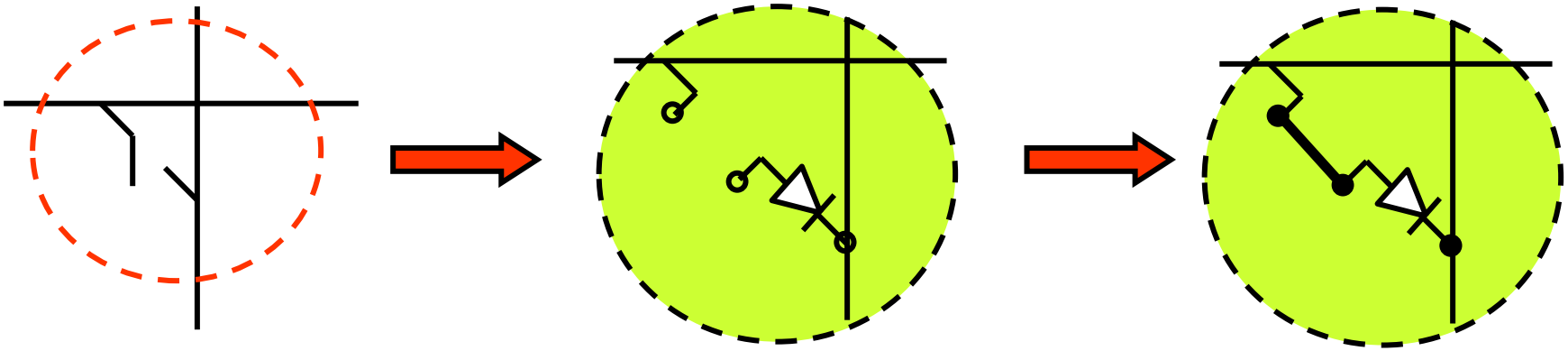
Byte, KByte, etc..

- ROM 2^n x k come circuito di **memoria** in grado di memorizzare $\mathbf{2^n}$ dati, ciascuno di $\boldsymbol{k}$ bit
- Tipicamente
  - $k = 8$ ovvero **1 byte**
  - n può variare da 10 (1KByte) a 20-22 (1-4 MB)

n	10	11	12	13	14	15	…	20
Byte	1024	2048	4096	8192	16384	32768		1.048.576
Taglia	1KB	2KB	4KB	8KB	16KB	32KB		1MB

La programmazione delle ROM

• I contatti vengono realizzati **dal costruttore** nell'ultimo stadio del processo di fabbricazione del circuito su ordine dell'acquirente:



- Le ROM sono programmabili **una sola volta** (OTP: one time programmable) e sono quindi **memorie a solo lettura (Read Only Memory) non volatili**

- Utili per la memorizzazione di informazioni che devono permanere quando il sistema non è alimentato e che non non cambiano durante il funzionamento (es. programma di un sistema «embedded», BIOS del PC, ..)
- Limiti: non riprogrammabili -> non adatte allo sviluppo, necessari grossi volumi di produzione, lunghi tempi di realizzazione

Memorie non volatili, a sola lettura e programmabili

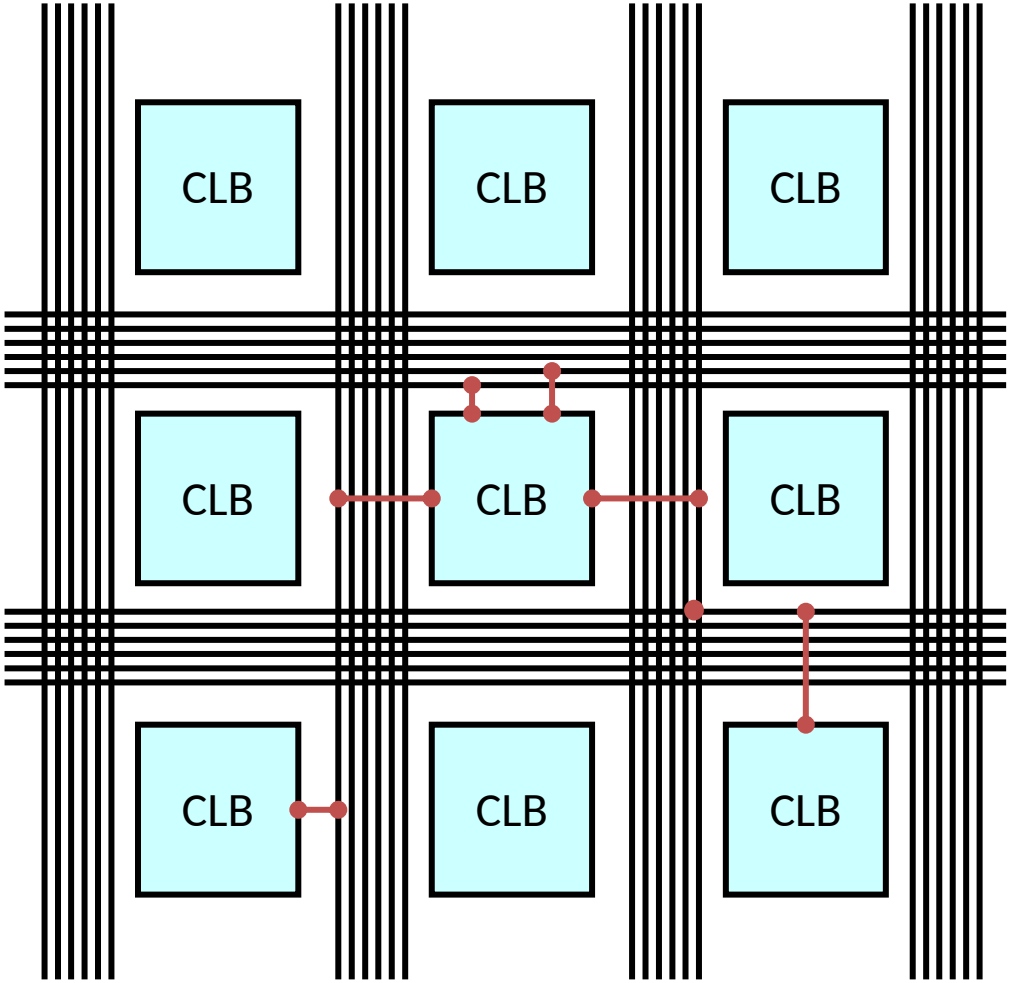
Negli anni sono state rese disponibili versioni più flessibili:

- Programmable ROM (PROM): **programmabili dall'utente una sola volta**
- Erasable PROM (EPROM): **ripogrammabili**, ma devono essere **tolte dal circuito** per cancellarne il contenuto e riscriverlo
- Electrically EPROM (EEPROM), Flash memory: riprogrammabili in modo elettrico **senza muoverle dal circuito**

Tipo	Programmabili			Uso
ROM	off line	una volta	dal costruttore	molte copie
PROM	off line	una volta	dall'utente	poche copie
EPROM	off line	più volte	dall'utente	prototipi
EEPROM, FLASH	on line	più volte	dall'utente	personale

CPLD, FPGA

- EEPROM interpretate come generatori di funzioni sono esempi di **blocchi logici riconfigurabili (Configurable Logic Blocks, CLB)**
- **Cambiando i bit di programmazione possiamo sintetizzare funzioni combinatorie diverse**
- La loro evoluzione ha portato a rendere disponibili al progettista circuiti più avanzati: **Complex PLD, Field Programmable Gate Array (FPGA)**
- In un FPGA anche le **connessioni tra i CLB sono programmabili**



Hardware Description Language (HDL)

- I circuiti programmabili e le connessioni in CPLD e FPGA sono configurati a partire da file scritti in cosiddetti **linguaggi di descrizione dell'hardware.**
- Si tratta a tutti gli effetti di linguaggi di programmazione specializzati per gestire esplicitamente la **concorrenza** intrinseca a livello HW e la **nozione di tempo**
- Dalla descrizione del comportamento fornita in HDL, un **sintetizzatore** realizza una sintesi mappabile sui gate, le altre risorse e le connessioni disponibili in uno specifico FPGA
- I due principali HDL sono
  - VHDL
  - Verilog

Esempio: VHDL per definire un MUX a 2 vie

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_2to1 is
    Port ( A : in  STD_LOGIC;      -- address
           I0  : in  STD_LOGIC;      -- input
           I1  : in  STD_LOGIC;      -- input
           Z   : out STD_LOGIC);     -- output
end mux_2to1;

architecture Behavioral of mux_2to1 is
begin
    Z <= I0   when (A = '0') else
         I1;
end Behavioral;
```

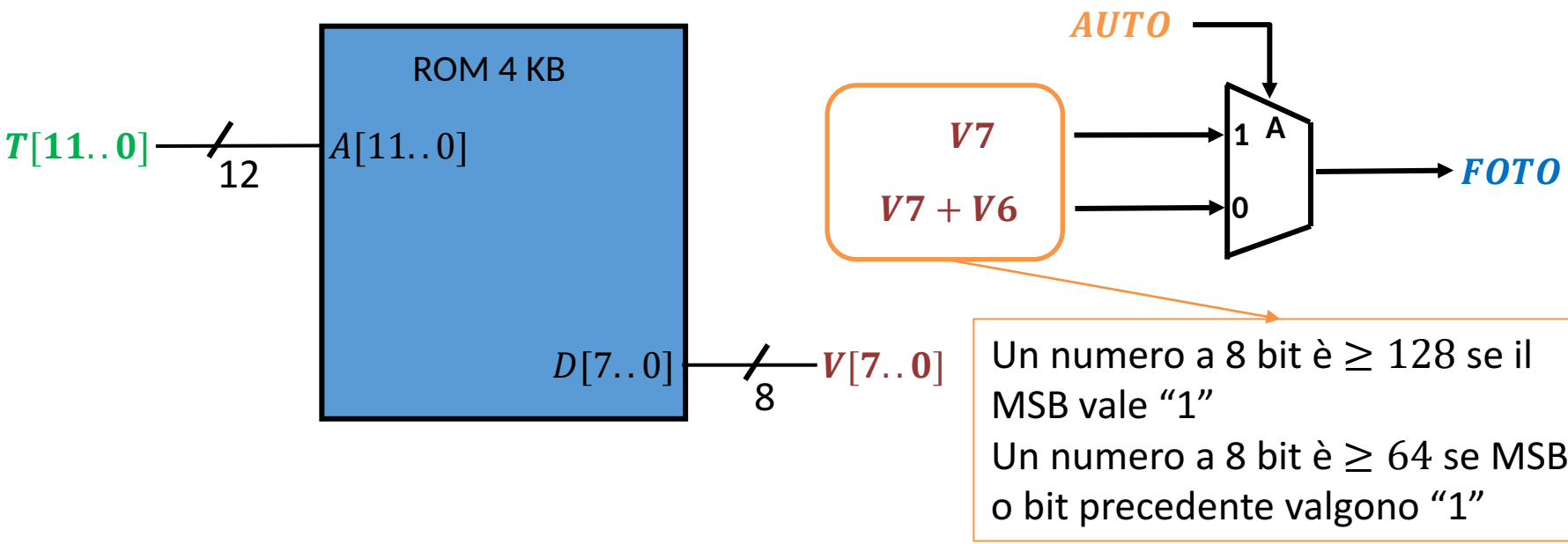
Esercizio

Progettare una rete combinatoria che realizza una parte del funzionamento di un autovelox a due fotocellule. In particolare, la rete riceve in ingresso sul bus **T[?..0]** il tempo in millisecondi trascorso tra l'attivazione della prima fotocellula e la seconda e l'indicazione se il veicolo è un'auto o un TIR tramite il segnale **AUTO**. Dal tempo è possibile ricavare, nota la distanza tra le fotocellule, la velocità del veicolo in km/h: tale funzione è realizzata da una ROM con 8 bit in uscita (1 byte). La rete deve attivare l'uscita **FOTO** se la velocità è maggiore o uguale a 128 km/h e il veicolo è un'auto, o è maggiore o uguale a 64 km/h e il veicolo è un TIR.

Indicare la dimensione del bus **T[?..0]** sapendo che la ROM che memorizza la conversione da tempo a velocità è da 4 KB. Qual è il massimo numero di millisecondi che il bus può rappresentare?

Soluzione

Una ROM da 4KB ha 12 bit di indirizzo, quindi il bus che codifica il tempo è **T[11..0]**. Il numero massimo di millisecondi che può riportare è $2^{12} - 1 = 4095$.



Esempio con 5 bit

Numero	$N_4N_3N_2N_1N_0$
0	00000
1	00001
2	00010
3	00011
4	00100
5	00101
6	00110
7	00111
8	01000
9	01001
10	01010
11	01011
12	01100
13	01101
14	01110
15	01111

Numero	$N_4N_3N_2N_1N_0$
16	10000
17	10001
18	10010
19	10011
20	10100
21	10101
22	10110
23	10111
24	11000
25	11001
26	11010
27	11011
28	11100
29	11101
30	11110
31	11111

Numero ≥ 16 se $N_4 = 1$
(Intervallo 16-23 e 24-31)

Numero ≥ 8 se $N_3 = \text{"1"}$
(Intervallo 8-15 e 24-31)
oppure se $N_4 = \text{"1"}$
(Intervallo 16-23 e 24-31)